

LESSON 9

Similarity in Artificial Intelligence Coding with Similarity





Lesson outline

- Image hash in Colab
- Concept of Cross-Correlation
- Matlab first steps with images
- Coding an example for similarity
 - Matlab
 - Python (image hash library)
- Image retrieval systems for specific image database creation
- Scaling laws for neural language models
- **Main points**

Knowing **what to learn** from the coding lesson?

- Matlab and Colab **first steps with images**
- Understanding the structure and steps of the coding example for the image similarity
- During the exam will be **no exact coding**
- It necessary to understand the main procedures (+ what are inputs and outputs)
- For example, today we will better understand the **image hashing** and **cross-corr.**



Knowing what to learn from the coding lesson?

It's not just about code!

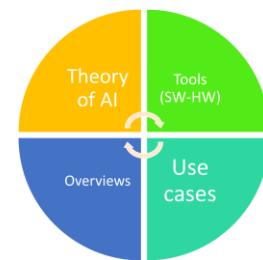
- We are learning how to find and identify objects, or just "similar patches" in an image.
- Understand the input format
 - Check image types, channels, pixel values.
 - Is it RGB or grayscale?
 - uint8 or float?
 - What do libraries really implement?
- Understand how the correlation (or normalized correlation) is implemented in libraries like OpenCV, SciPy, or MATLAB.
 - Do they normalize?
 - Do they subtract the mean?
- Signal vs. Background Understand when your filter (template) is matching the background, not the object.
- Use plotting to validate results: Always visualize!



Toolboxes

Colab

Processing image hash



A very simple example of image hash in Colab

Lesson10_SimilarityInImages_imagehash.ipynb

```
!pip install imagehash
```

A powerful library easy to be used

Project description

An image hashing library written in Python. ImageHash supports:

- Average hashing
- Perceptual hashing
- Difference hashing
- Wavelet hashing
- HSV color hashing (colorhash)
- Crop-resistant hashing

```
>>> from PIL import Image
>>> import imagehash
>>> hash = imagehash.average_hash(Image.open('tests/data/imagehash.png'))
>>> print(hash)
ffd7918181c9ffff
```

```
>>> print(hash - otherhash) # hamming distance
```

```
33
```

Download the images

```
# import openCV to load and manage the images
import cv2
```



<https://opencv.org/>

```
# download image from web
import requests
import os

with open('dog1.jpg', 'wb') as handler: handler.write(requests.get('https://scotti.di.unimi.it/img2/dog1.jpg').content)
with open('dog2.jpg', 'wb') as handler: handler.write(requests.get('https://scotti.di.unimi.it/img2/dog2.jpg').content)
with open('dog3.jpg', 'wb') as handler: handler.write(requests.get('https://scotti.di.unimi.it/img2/dog3.jpg').content)
with open('dog4.jpg', 'wb') as handler: handler.write(requests.get('https://scotti.di.unimi.it/img2/dog4.jpg').content)
with open('panda.jpg', 'wb') as handler: handler.write(requests.get('https://scotti.di.unimi.it/img2/panda.jpg').content)

# let's see what we got
!ls
```

```
dog1.jpg  dog2.jpg  dog3.jpg  dog4.jpg  panda.jpg  sample_data
```

Load the images

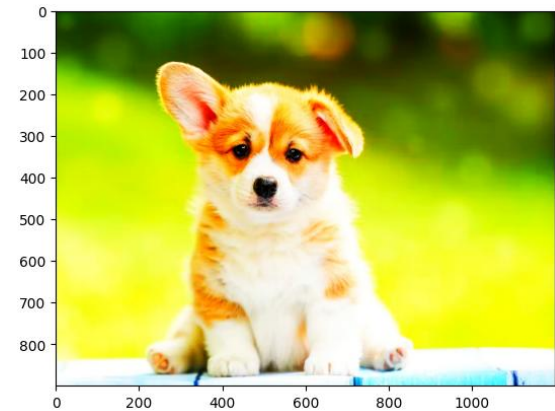
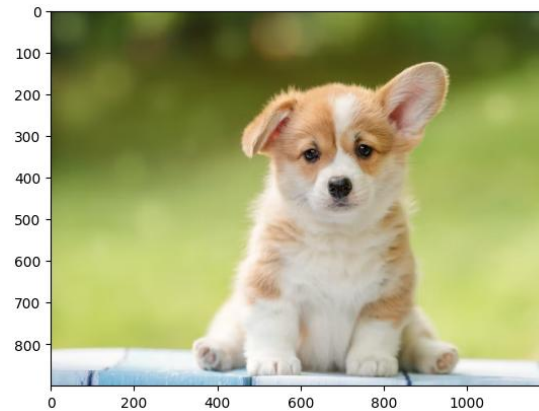
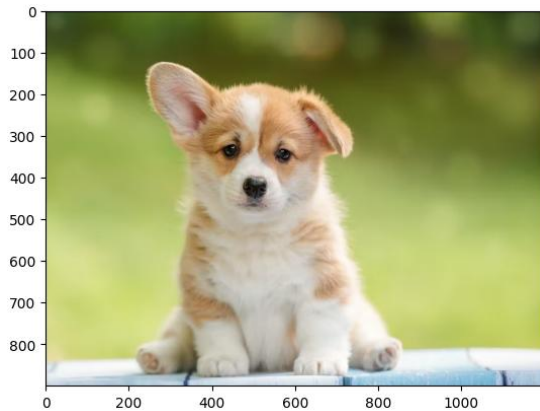
(+ conversion in the proper colorspace)

```
#load and plot the downloaded images
from matplotlib import pyplot as plt

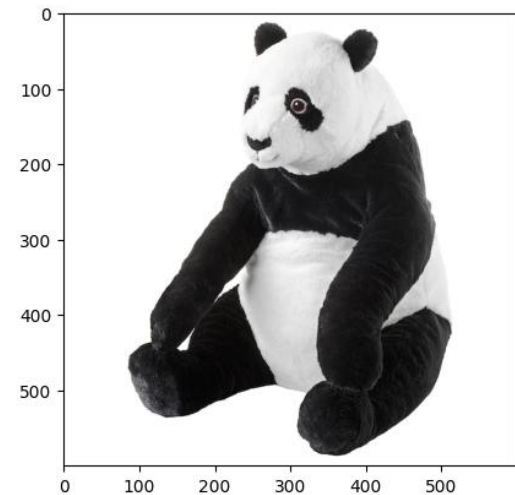
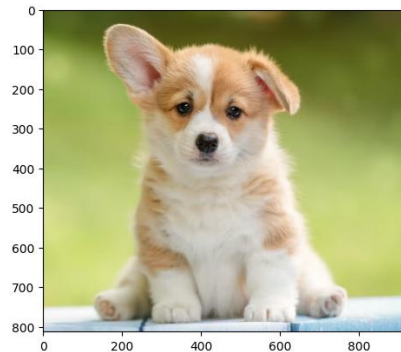
img_color1 = cv2.imread('dog1.jpg')
img_color1 = cv2.cvtColor(img_color1, cv2.COLOR_BGR2RGB)

img_color2 = cv2.imread('dog2.jpg')
img_color2 = cv2.cvtColor(img_color2, cv2.COLOR_BGR2RGB)
img_color3 = cv2.imread('dog3.jpg')
img_color3 = cv2.cvtColor(img_color3, cv2.COLOR_BGR2RGB)
img_color4 = cv2.imread('dog4.jpg')
img_color4 = cv2.cvtColor(img_color4, cv2.COLOR_BGR2RGB)
img_color5 = cv2.imread('panda.jpg')
img_color5 = cv2.cvtColor(img_color5, cv2.COLOR_BGR2RGB)
```

Plot the images



```
plt.imshow(img_color1)
plt.show()
plt.imshow(img_color2)
plt.show()
plt.imshow(img_color3)
plt.show()
plt.imshow(img_color4)
plt.show()
plt.imshow(img_color5)
plt.show()
```



Processing the image ahash

aHash, pHash, dHash

- 3 methods with similar initial steps
- Looking at average, perceptive and gradient difference and creating a binary feature
- aHash (a = Average)
 - 1) Reduce size (example: 8x8 R,G,B)
 - 2) Reduce to gray (8x8)
 - 3) Reduce to bits (0 if gray > below the mean gray)
 - 4) Merge the bits into a 64-bit integer. ← your hash

```
#----- AVERAGE HASH-----
hash1 = imagehash.average_hash(Image.open('dog1.jpg'))
hash2 = imagehash.average_hash(Image.open('dog2.jpg'))
hash3 = imagehash.average_hash(Image.open('dog3.jpg'))
hash4 = imagehash.average_hash(Image.open('dog4.jpg'))
hash5 = imagehash.average_hash(Image.open('panda.jpg'))
```

```
print(hash1)
print(hash2)
print(hash3)
print(hash4)
print(hash5)
```

```
print('\n')
print('original - flipped:')
print(hash1 - hash2)
```

```
print('original - saturated:')
print(hash1 - hash3)
```

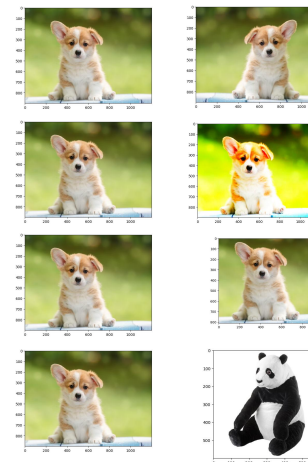
```
print('original - cropped:')
print(hash1 - hash4)
```

```
print('original - panda:')
print(hash1 - hash5)
```

Hamming
distance
(bit2bit)

```
0000d0e8fffffffff
00000b17fffffffff
0000f0ecfffffffff
001040ecffdfdfdf
ffffe3e1c98181f3
```

```
original - flipped:
14
original - saturated:
2
original - cropped:
7
original - panda:
40
```

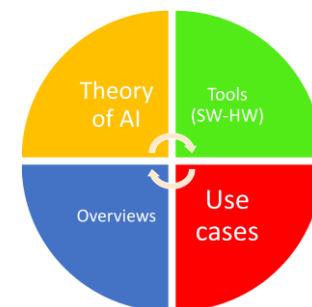
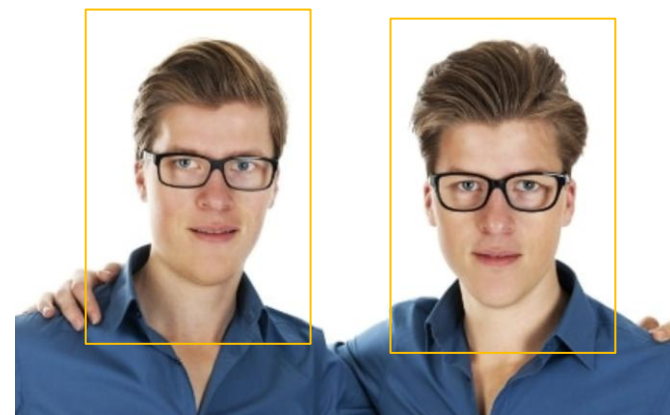


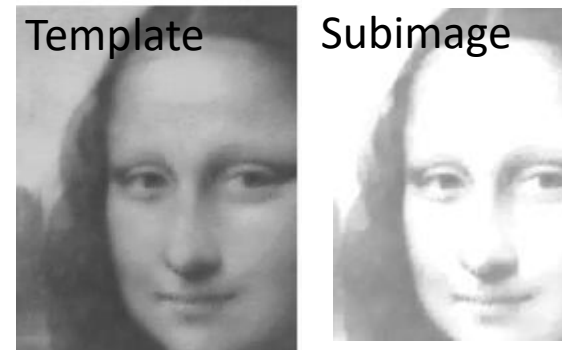


THEORY

Similarity in images via cross-correlation

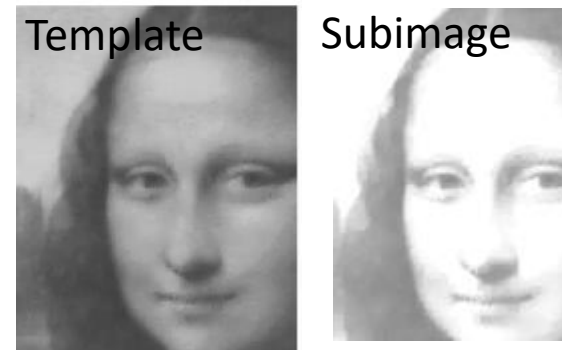
What is relevant what is useless?





- Is a measure of similarity of two vectors (matrices) as a function of the displacement of one relative to the other.
- Cross-correlation returns
 - the **similarity** value
 - and the position with **the better alignment** of the two images.

`[similarity, displacement] = image_xcorr( I1, I2 )`



Similarity by Cross-correlation

- The “**core** of cross-correlation”:
for image-processing applications in which the brightness of the image and template can vary due to lighting and exposure conditions, **the images can be first normalized**

- This is typically done at every step by subtracting the mean and dividing by the standard deviation.

Template = $t(x,y)$, Subimage $f(x,y)$

Repeat
shifting the
subimage f to get
the alignment points
checking where is
the maximum value
of NCC

Normalized
Cross-Correlation = $t * f = \frac{1}{n} \sum_{x,y} \frac{1}{\sigma_f \sigma_t} (f(x,y) - \mu_f) (t(x,y) - \mu_t).$
(NCC)

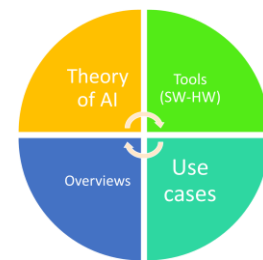
$f(x+i,y+j)$



Toolboxes

Matlab

First step with images and similarity



Why similarity is **so important**?

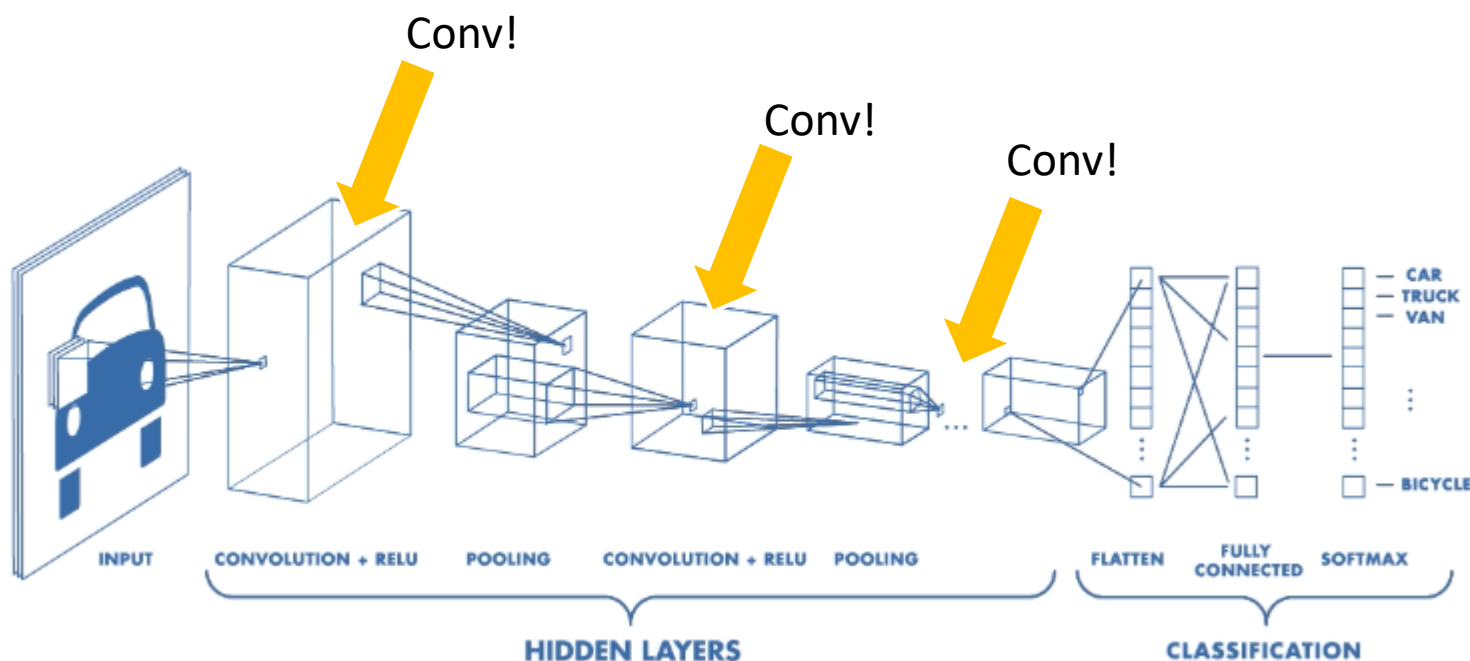


- Because it is one of the basis/kernel/primitive of the **pattern recognition** and **machine learning**
- **You/The Intelligent system can recognize a pattern/object/vector/image because it is similar to a previously labelled one seen the learning phase**

Why similarity is so important?



Convolution $\leftrightarrow$ Correlation Convolutional Neural Networks



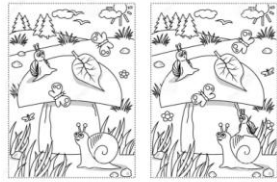
Have you noticed the puzzle?

(Lesson 9)

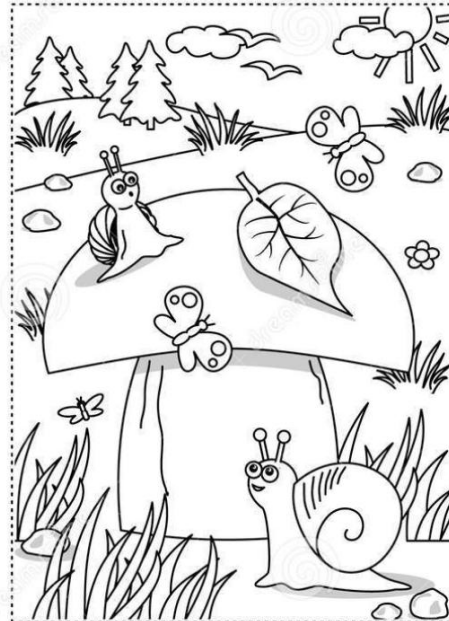
Image Similarity

- **Image Similarity** compares two images and returns a value that tells you how visually similar they are.
`score = similarity(image1, image2)`
- If Score $\rightarrow 0$ so images $\rightarrow$ Contextually similar
- If (score == 0) so images are identical

Fabio Scotti - Università degli Studi di Milano



Did you find all the differences?
Are you sure?

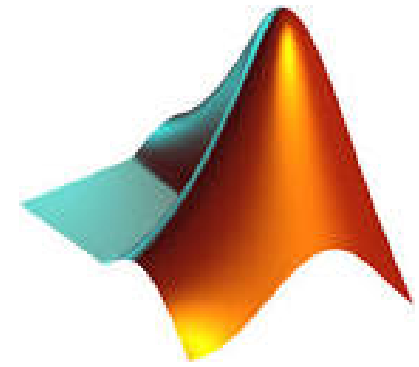


How to automate the task?

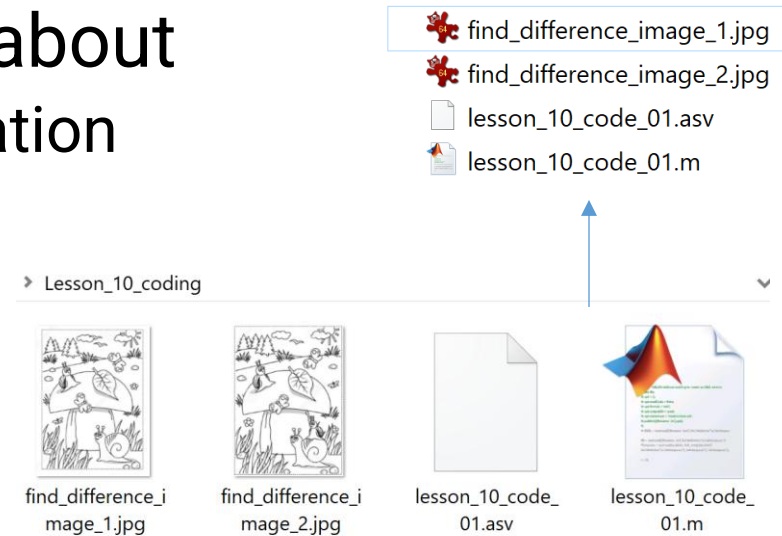
All what we need is

- Concept of image cross-correlation
- ...and a little of matlab coding

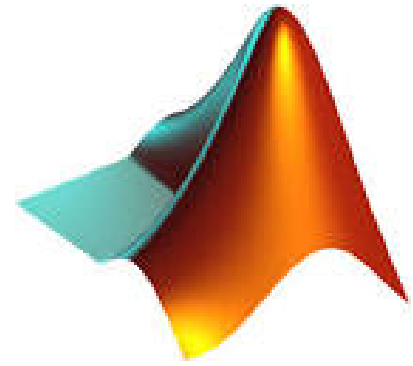
The material you need



- Matlab (last version)
- Download the files of the lesson in local folder
- Do not jump to the solution....
- Follow the process,
you will gather information about
 - Image format and representation
 - Image manipulation
 - Image plotting
 - Image similarity



Files



- In the ariel site you will find a set of files similar to this one



find_difference_i
mage_1.jpg



find_difference_i
mage_2.jpg



find_difference_i
mage_3.jpg



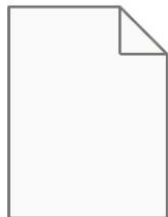
ice_creams.jpg



icecream2.jpg



lesson_10_code_
01.m



lesson_10_code_
02.asv



lesson_10_code_
02.m



lesson_10_code_f
inding_similarity.
m



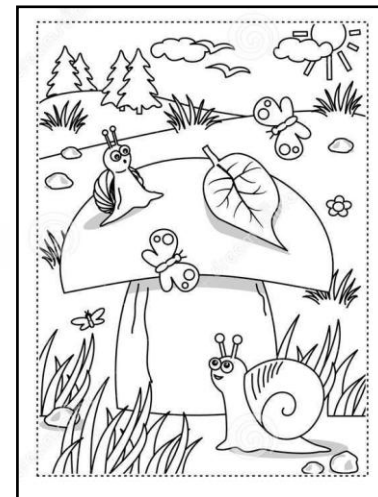
panda.jpg

Two steps

STEP #1

Understanding how to use the convolution/correlation to find similarity in images

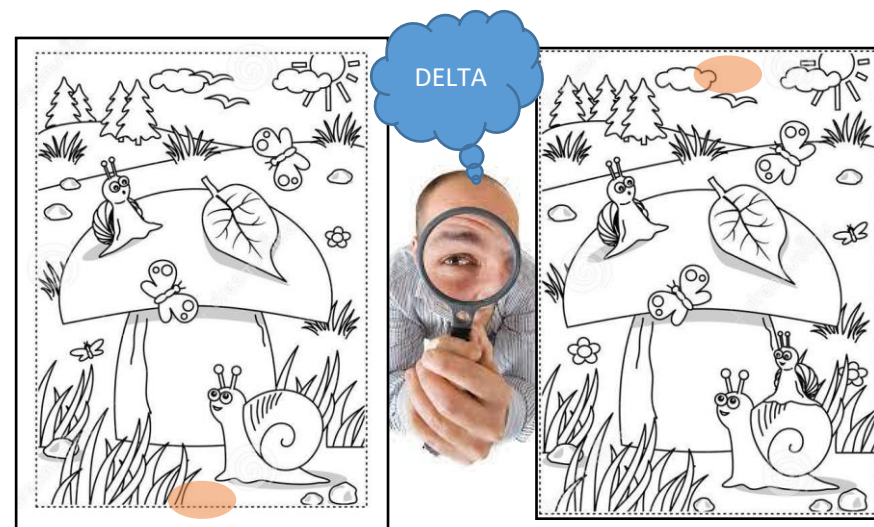
(we want to find a leaf)



STEP #2

Solving the puzzle by a completely automated approach by coding using similarity

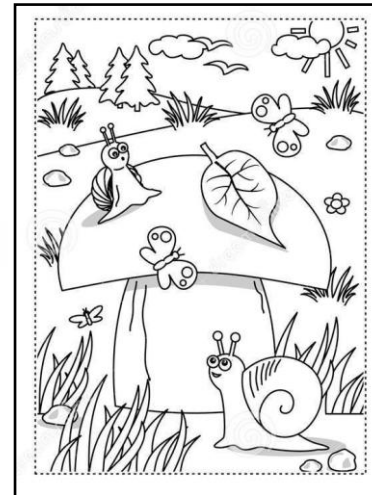
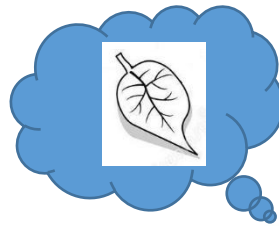
(we want to find the differences)



STEP #1

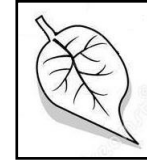
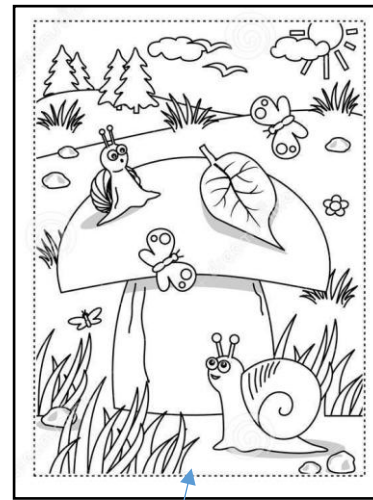


lesson_10_code_
STEP1.m



(we want to find a leaf)

Loading the files



```
img_template = imread('find_difference_image_1.jpg');  
img_subimage = imread('find_difference_image_3.jpg');
```

whos

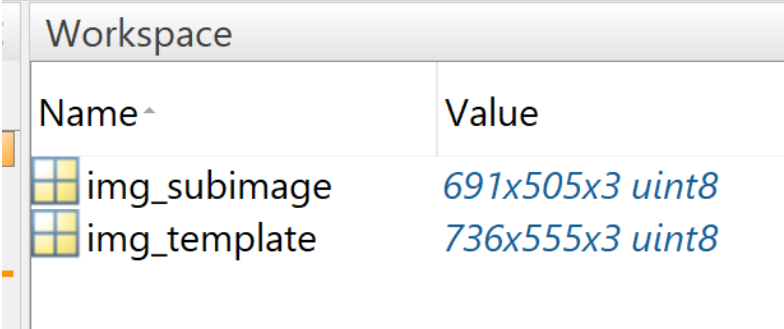
Name	Size	Color image (RGB)	Bytes	Class	Attributes
img_subimage	193x163x3		94377	uint8	
img_template	736x555x3		1225440	uint8	

Unsigned 8-bit integers

The uint8 range is from 0 to 255

Have a look to the variables

- The workspace
- `>> whos`

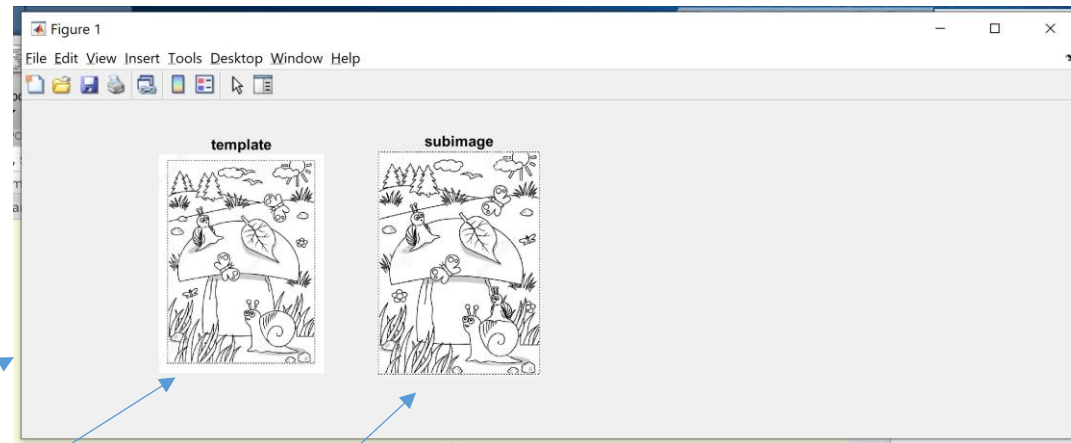


The image shows a screenshot of the MATLAB Workspace window. It has a title bar 'Workspace' and a table with two columns: 'Name' and 'Value'. There are two rows of variables: 'img_subimage' with value '691x505x3 uint8' and 'img_template' with value '736x555x3 uint8'. Each variable name has a small icon to its left.

Name	Value
img_subimage	691x505x3 uint8
img_template	736x555x3 uint8

Name	Size	Bytes	Class
img_subimage	691x505x3	1046865	uint8
img_template	736x555x3	1225440	uint8

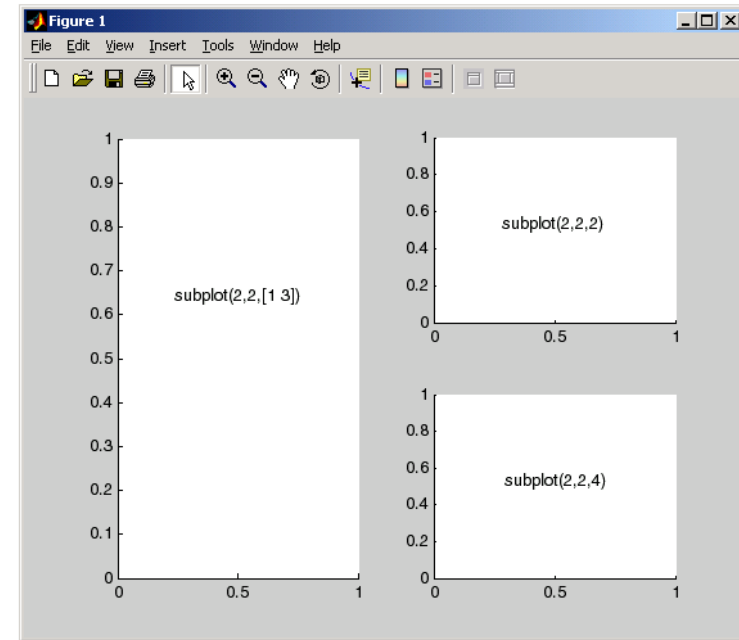
Plotting images



handle

```
h1 = figure;  
subplot(1,4,1)  
imshow(img_template, [])  
title('template')
```

```
subplot(1,4,2)  
imshow(img_subimage, [])  
title('subimage')
```



Plotting images (2)



```
h1 = figure;  
subplot(1,4,1)  
imshow(img_template, [])  
title('template')
```

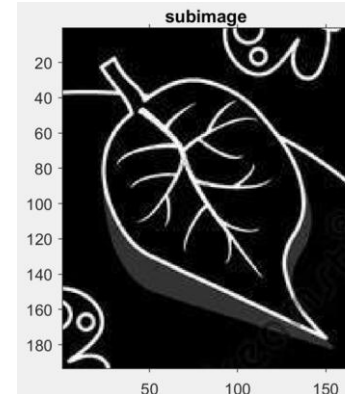
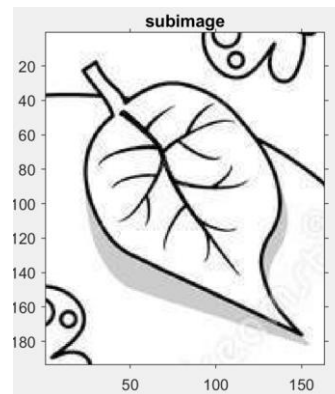
```
subplot(1,4,2)  
imshow(img_subimage, [])  
title('subimage')
```

Auto color scale (**Square brackets**)

imshow(I,[]) displays the grayscale image I scaling the display based on the range of pixel values in I. **imshow** uses [min(I(:)) max(I(:))] as the display range, that is, the minimum value in I is displayed as black, and the maximum value is displayed as white.

To Invert or Not invert the images? That is the problem

- Where is the information in an image?
Is it in the
 - Colors?
 - Patterns?
 - White patterns? Black patterns?
- In the convolutions will contribute to the sum the high value of the matrices
- Here the high values are associated to the white background (less useless)
 - → Let's invert the image!



RGB → Gray

+ Inverting the images

```
%% color 2 gray conversion
```

```
img_template_gray = rgb2gray( img_template );
```

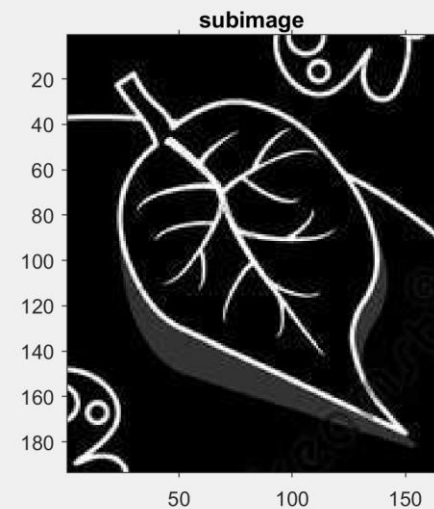
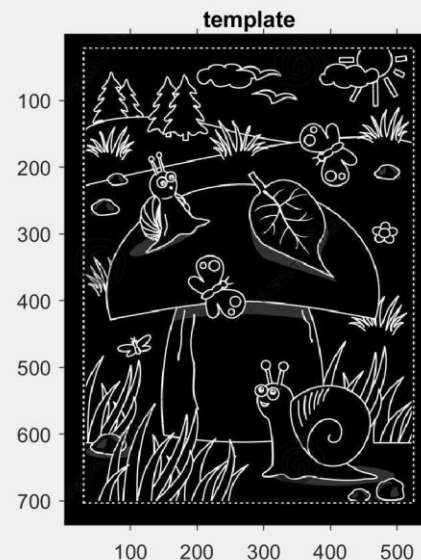
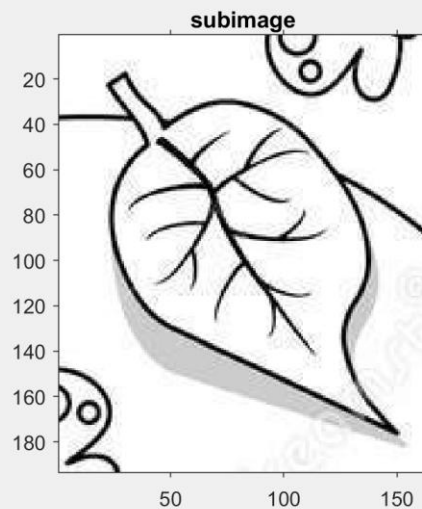
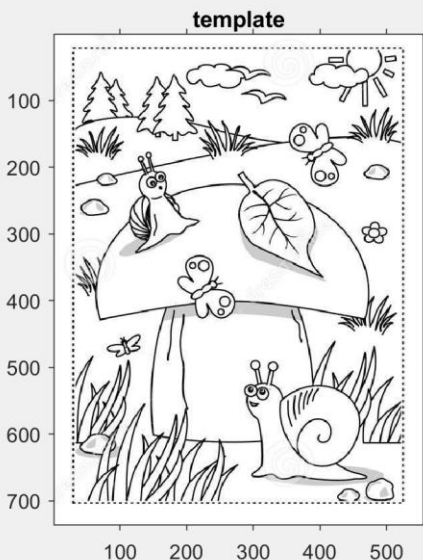
```
img_subimage_gray = rgb2gray( img_subimage );
```

It's better to invert the image to grasp more signal from the drawing (now white) and not from the background (black)

```
%% inversion
```

```
img_template_gray = max(img_template_gray) - img_template_gray ;
```

```
img_subimage_gray = max(img_subimage_gray) - img_subimage_gray ;
```



Plotting the images

```
h1 = figure;
subplot(1,4,1)
imshow(img_template, [])
axis on
title('template')
```

```
subplot(1,4,2)|
imshow(img_subimage, [])
axis on
title('subimage')
```

```
figure (h1);
subplot(1,4,3)
imshow(img_template_gray, [])
axis on
title('template')
```

```
subplot(1,4,4)
imshow(img_subimage_gray, [])
axis on
title('subimage')
```

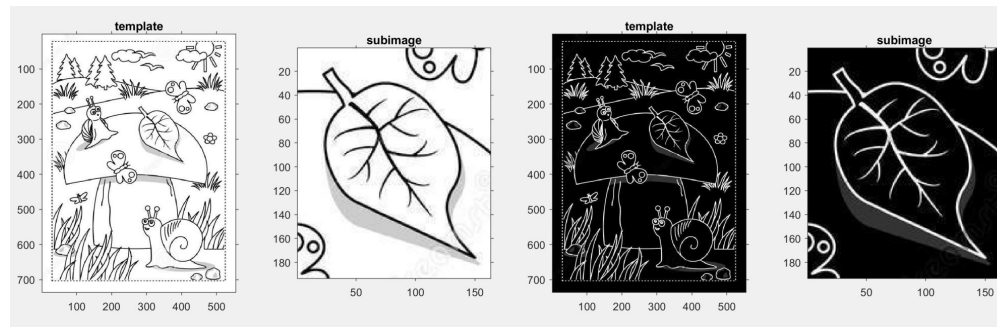


Image normalization (just mean subtraction)

```
img_template_gray_norm = img_template_gray - mean(mean(img_template_gray)) ;  
img_subimage_gray_norm = img_subimage_gray - mean(mean(img_subimage_gray)) ;
```

Just without the means (no standard deviations)

Two times..

mean Average or mean value.

$S = \text{mean}(X)$ is the mean value of the elements in X if X is a vector.
For matrices, S is a row vector containing the mean value of each column.

For N-D arrays, S is the mean value of the elements along the first array dimension whose size does not equal 1.

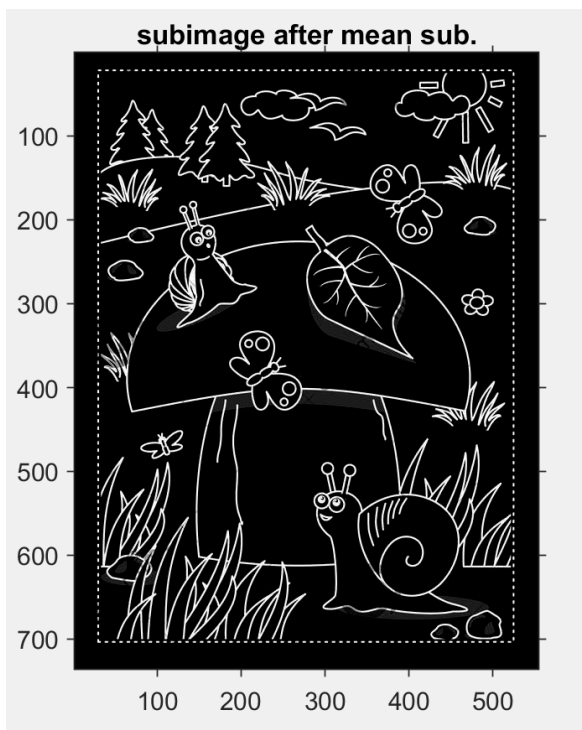
Image normalization (just mean subtraction)

```
img_template_gray_norm = img_template_gray - mean(mean(img_template_gray)) ;  
img_subimage_gray_norm = img_subimage_gray - mean(mean(img_subimage_gray)) ;
```

Just without the means (no standard deviations used here)

Similar operator

```
figure  
imshow(img_template_gray, [])  
axis on  
title('subimage after mean sub.')  
mean_val = mean(img_template_gray(:))  
min_val = min(img_template_gray(:))  
max_val = max(img_template_gray(:))
```



It's quite dark!

white == 255
@uint8

mean_val = 20.2077
min_val = 0
max_val = 229

2D correlation (similarity evaluation)

GENERAL

Normalized cross-correlation = (NCC)

$$t * f = \frac{1}{n} \sum_{x,y} \frac{1}{\sigma_f \sigma_t} (f(x, y) - \mu_f) (t(x, y) - \mu_t).$$

MATLAB → with no scaling (direct!)

```
% xcorr2(A,B) computes the crosscorrelation of matrices A and B.
crr = xcorr2(img_template_gray_norm , img_subimage_gray_norm);
```

2-D Cross-Correlation

The 2-D cross-correlation of an M -by- N matrix, X , and a P -by- Q matrix, H , is a matrix, C , of size $M+P-1$ by $N+Q-1$.

$$C(k, l) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X(m, n) \overline{H(m-k, n-l)},$$

$$\begin{aligned} -(P-1) &\leq k \leq M-1, \\ -(Q-1) &\leq l \leq N-1, \end{aligned}$$

2D correlation (similarity evaluation)

```
% xcorr2(A,B) computes the crosscorrelation of matrices A and B.
crr = xcorr2(img_template_gray_norm , img_subimage_gray_norm);
```

Normalized cross-correlation = (NCC)

$$t * f = \frac{1}{n} \sum_{x,y} \frac{1}{\sigma_f \sigma_t} (f(x,y) - \mu_f) (t(x,y) - \mu_t).$$

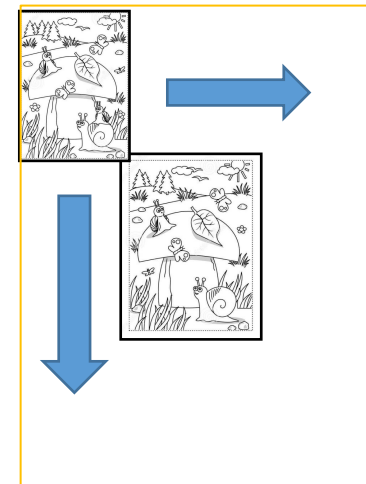
Repeat shifting the subimage f to get the alignment points checking where is the maximum value of NCC $f(x+i,y+j)$



```
>> whos crr
```

Name	Size	Bytes	Class	Attributes
crr	1426x1059	12081072	double	

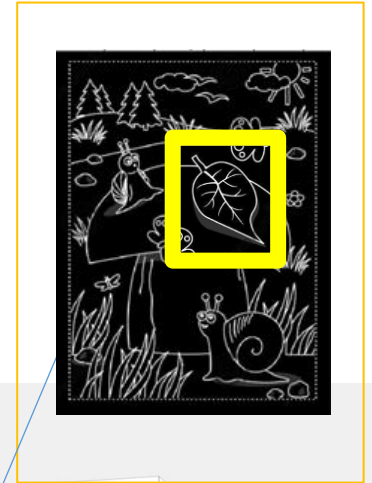
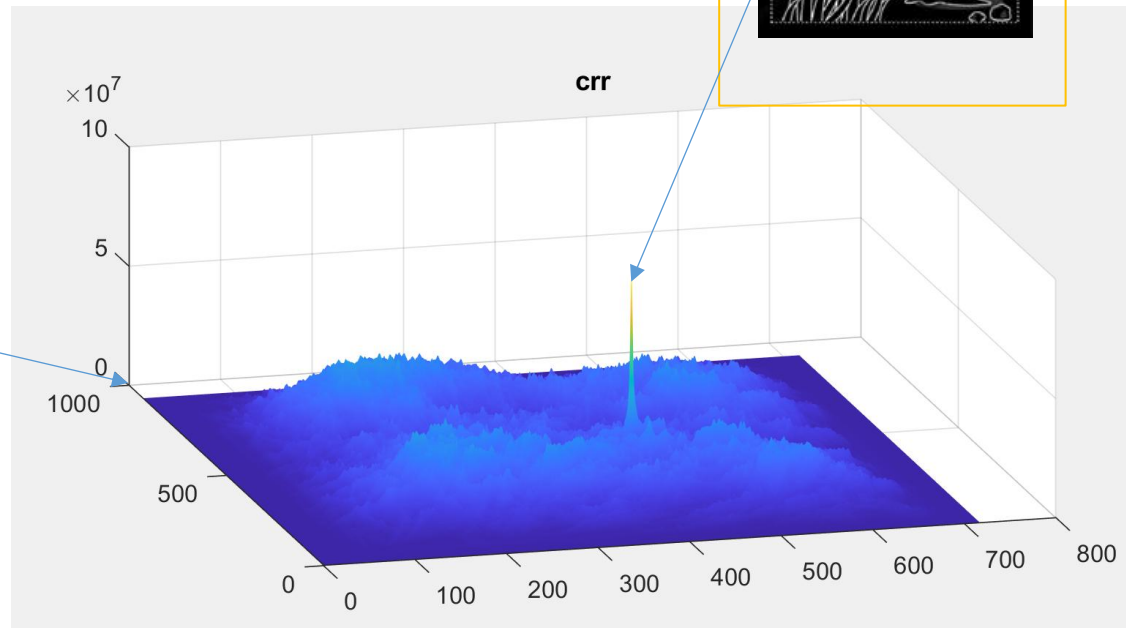
Name	Size	Bytes	Class
h1	1x1	8	matlab.ui.Figure
img_subimage	691x505x3	1046865	uint8
img_subimage_gray	691x505	348955	uint8
img_template	736x555x3	1225440	uint8
img_template_gray	736x555	408480	uint8



How is the output of a xcorr2?

```
%%  
figure  
surf(crr);  
shading interp;  
title('crr')
```

just to remove black
line around the tiles



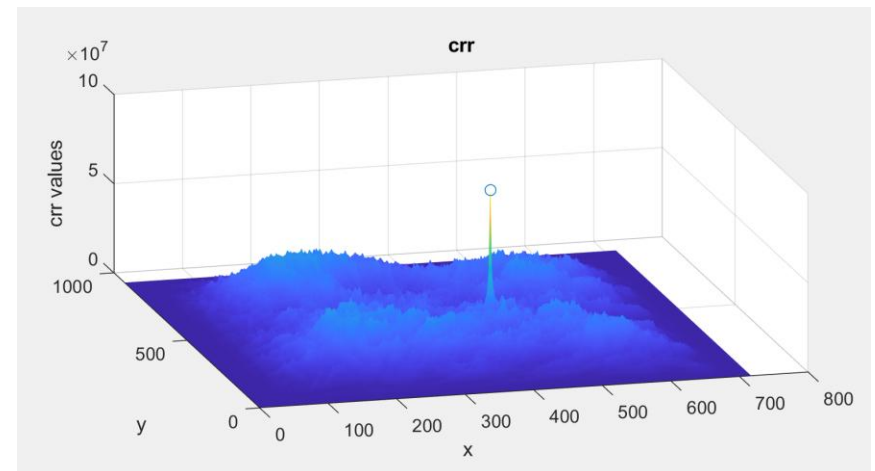
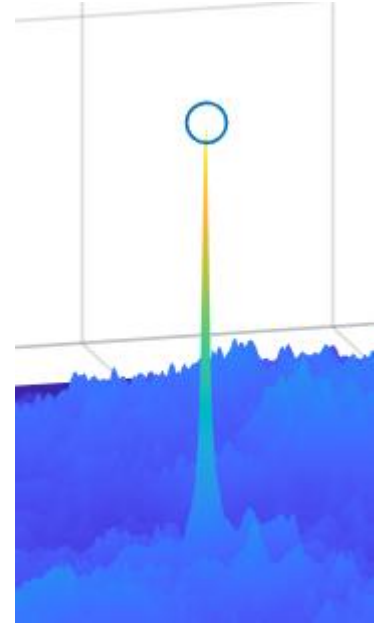
Find the maximum

```
% find the maximum  
[ssr,snd] = max( crr(:) );  
[ij,ji] = ind2sub(size(crr),snd);
```

```
%%  
hold on  
plot3( ji, ij, ssr, 'o')  
xlabel('x');  
ylabel('y');  
zlabel('crr values');
```

With similar images the peak is less spiky

This concept is used in convolutional NN to look for features with specific patterns in the input

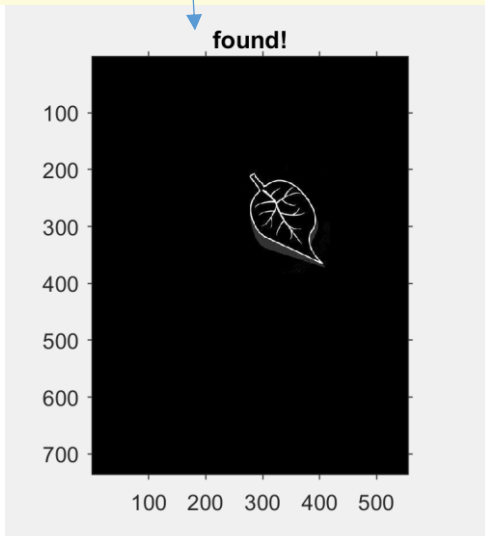


A final check

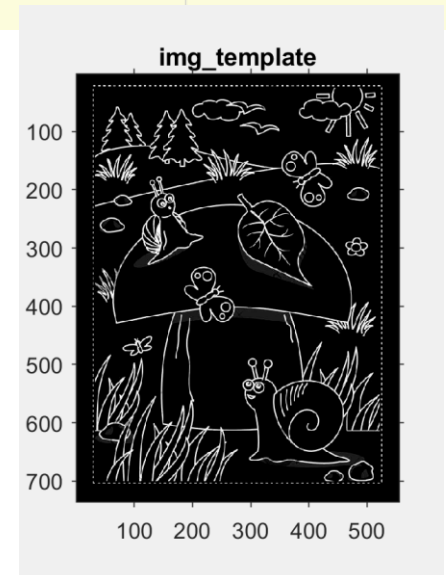
(the code is slightly more structured just for visualization, not in exam)

```
% creating a canvas full of zeros same size of the input image
img_diff = zeros(size(img_template_gray_norm)) ;
% placing in the ij e and ji coordinates the template image for comparison
img_diff(ij:-1:ij-size(img_subimage_gray,1)+1,ji:-1:ji-size(img_subimage_gray,2)+1) = ...
    rot90(img_subimage_gray,2) ;

subplot(2,2,4)
imshow(img_diff, []);
axis on
title('found!', 'interp' , 'none')
```



To help to understand is the output is correct, let's create a zero-based image and place the subimage centered in the maximum position of the correlation



Plotting all together

(the code is slightly more structured just for visualization, not in exam)

```
figure
subplot(2,2,1)
surf(crr);
shading interp; % just to remove black line around the tiles
title('crr (z=values)')

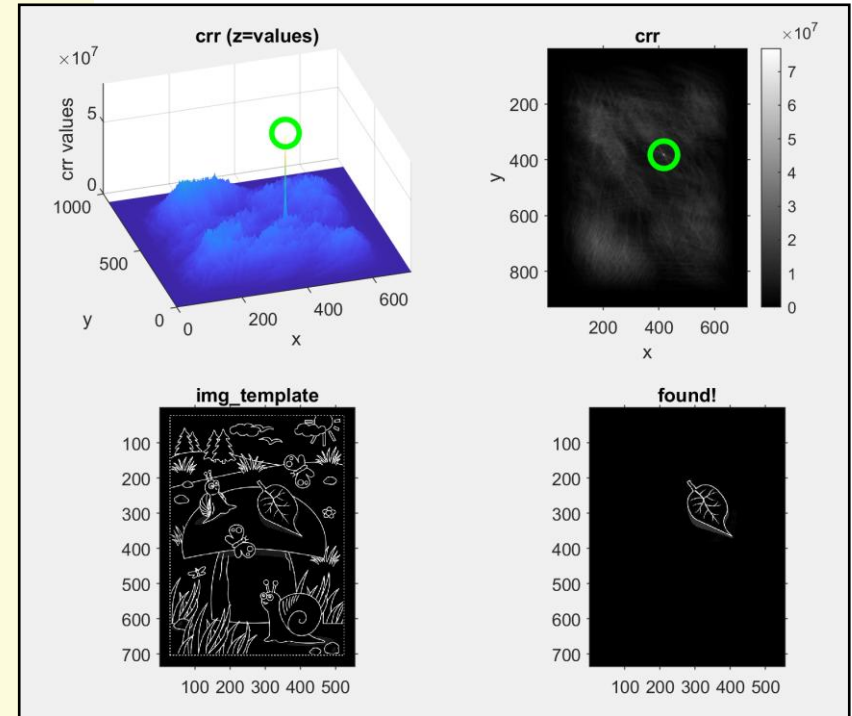
hold on % fix the plot to draw something later
plot3( ji, ij, ssr,'go', 'MarkerSize', 15, 'LineWidth',3)
xlabel('x');
ylabel('y');
zlabel('crr values');

subplot(2,2,2)
imshow(crr, []);
axis on
colorbar
title('crr')
hold on % fix the plot to draw something later
plot( ji, ij, 'go', 'MarkerSize', 15, 'LineWidth',3)
xlabel('x');
ylabel('y');

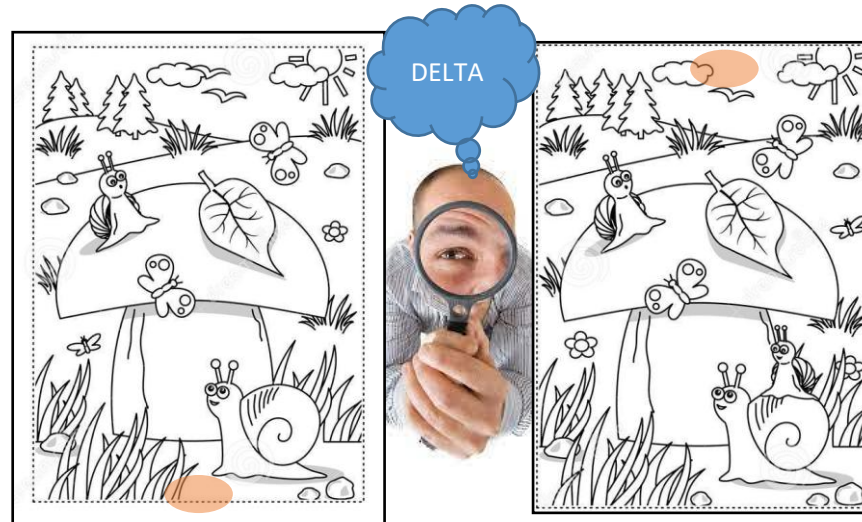
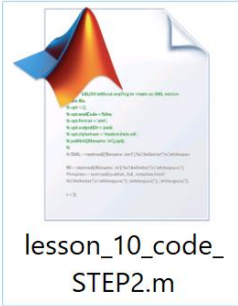
subplot(2,2,3)
imshow(img_template_gray_norm, []);
title('img_template', 'interp' , 'none')
axis on

% creating a canvas full of zeros same size of the input image
img_diff = zeros(size(img_template_gray_norm)) ;
% placing in the ij e and ji coordinates the template image for comparison
img_diff(ij:-1:ij-size(img_subimage_gray,1)+1,ji:-1:ji-size(img_subimage_gray,2)+1) = ...
    rot90(img_subimage_gray,2) ;

subplot(2,2,4)
imshow(img_diff, []);
axis on
title('found!', 'interp' , 'none')
```

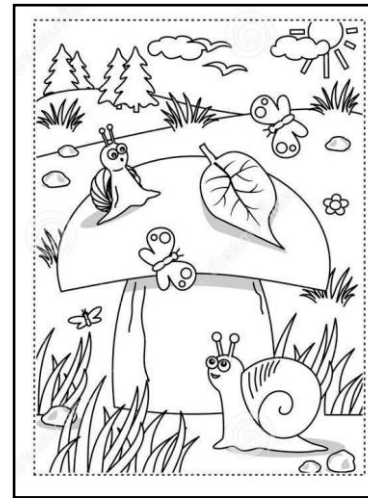
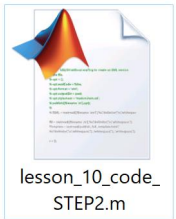


STEP #2



(we want to find the differences)

Loading the files



```
close all
clear all % do not use it in case of debug...

img_template = imread('find_difference_image_1.jpg');
img_subimage = imread('find_difference_image_2.jpg');
```

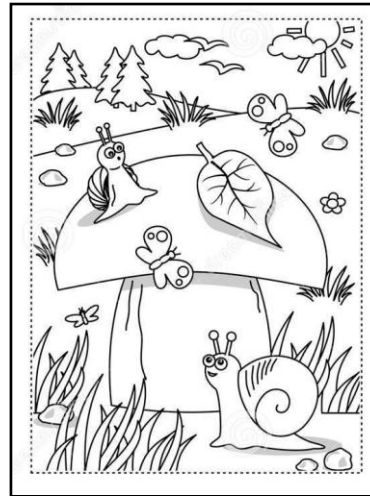
Name	Size	Bytes	Class
img_subimage	691x505x3	1046865	uint8
img_template	736x555x3	1225440	uint8

Color image
(RGB)

How to see the difference?

The general idea is to subtract two images to find the differences

We will use the correlation to find where to subtract them!



Name	Size	Bytes	Class
img_subimage	691x505x3	1046865	uint8
img_template	736x555x3	1225440	uint8

- 1) Images are different in size → we can't just subtract them (impossible in every coding language)
- 2) We have to place properly the second image to avoid false detections

Color conversion

```
% color 2 gray conversion
img_template_gray = rgb2gray( img_template );
img_subimage_gray = rgb2gray( img_subimage );
```

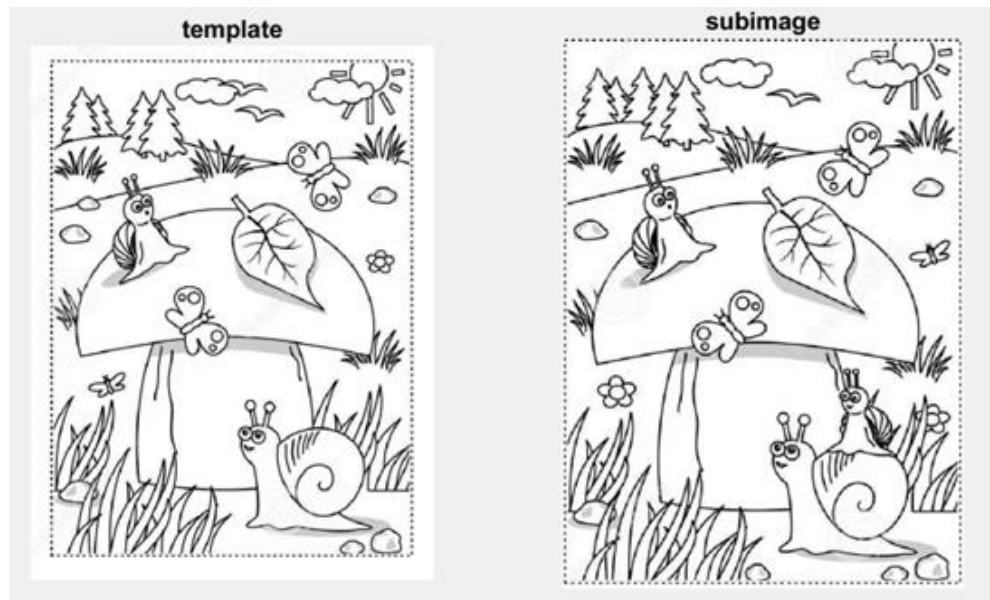
Name	Size	Bytes	Class
h1	1x1	8	matlab.ui.Figure
img_subimage	691x505x3	1046865	uint8
img_subimage_gray	691x505	348955	uint8
img_template	736x555x3	1225440	uint8
img_template_gray	736x555	408480	uint8

The handle of the figure points to h1.

gray images points to img_subimage_gray and img_template_gray.

Restart with the white images

```
close all  
clear all % do not use it in case of debug...  
  
img_template = imread('find_difference_image_1.jpg');  
img_subimage = imread('find_difference_image_2.jpg');
```



Same operations as before

```
% color 2 gray conversion
img_template_gray = rgb2gray( img_template );
img_subimage_gray = rgb2gray( img_subimage );

whos

img_template_gray_norm = img_template_gray - mean(mean(img_template_gray)) ;
img_subimage_gray_norm = img_subimage_gray - mean(mean(img_subimage_gray)) ;

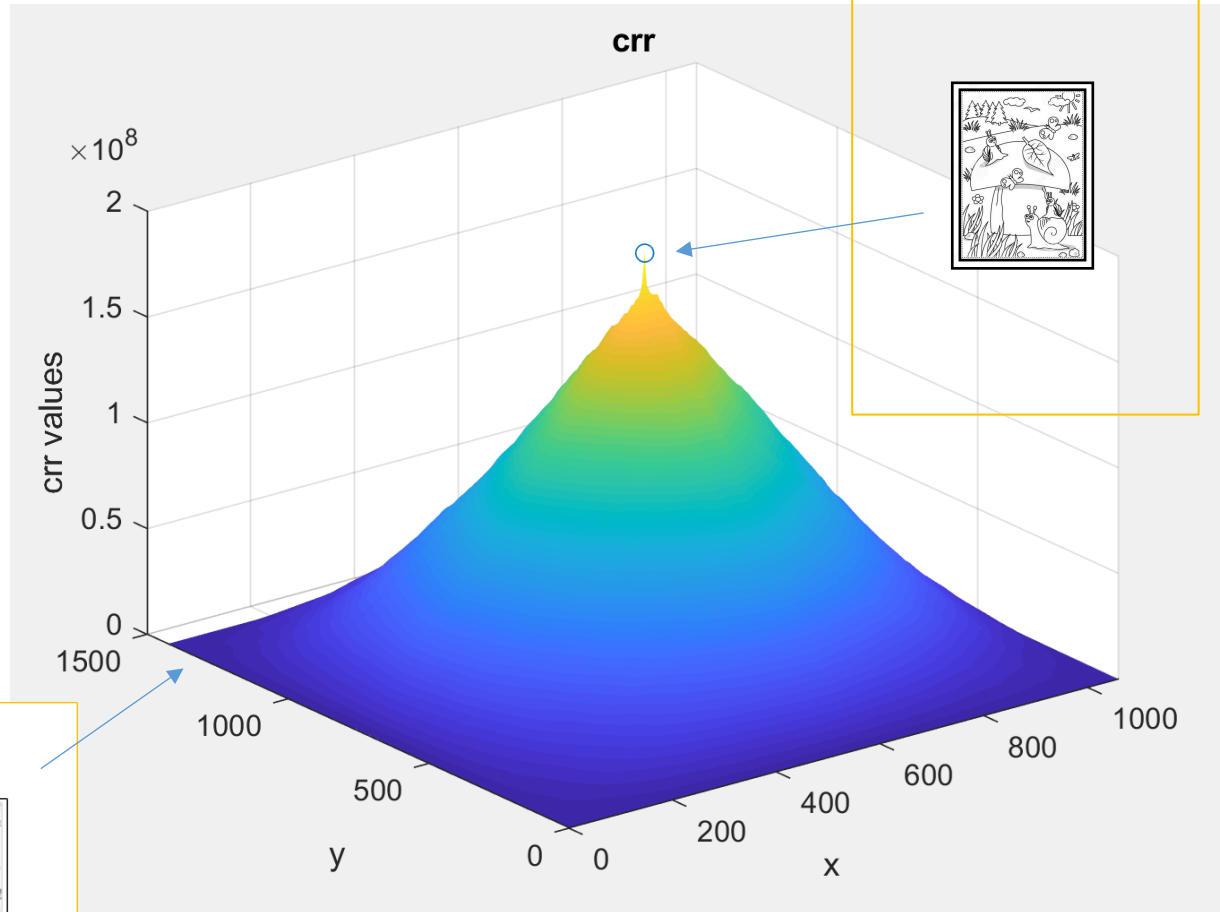
% xcorr2(A,B) computes the crosscorrelation of matrices A and B.
crr = xcorr2(img_template_gray_norm , img_subimage_gray_norm);
```

This time we try to not inverting the image.

How is the output of a xcorr2? (with white images)

```
%%  
figure  
surf(crr);  
shading interp;  
title('crr')
```

just to remove black
line around the tiles



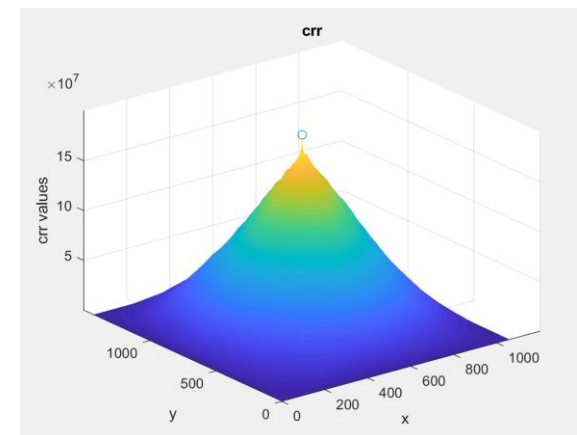
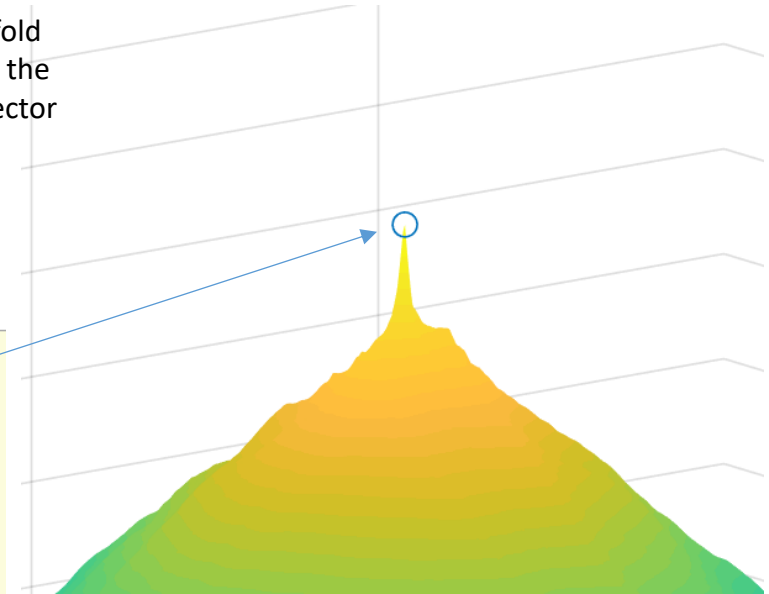
Note: with white images, the crosscorrelation is working the same,
but the peak is on a «big hill»

A spike! Locate the max in ccr!

```
% find the maximum
[ssr,snd] = max( crr(:) );
[ij,ji] = ind2sub(size(crr),snd);
```

operator (:) unfold
the elements of the
matrix in a 1D vector

```
%%
hold on
plot3( ji, ij, ssr, 'o')
xlabel('x');
ylabel('y');
zlabel('crr values');
```



See the difference?

→ Image subtraction (Part 1)

size

The basic idea is to see the differences by subtracting the original template with a proper image containing the subimage properly **sized** and **shifted**

$$\text{img_delta} = \text{template} - \text{img_diff}$$

```
% copy the original image (**just for the EXTERNAL FRAME**)
img_diff = img_template_gray ;
```



Template

This is just to

- 1) Get the proper **dimensions** since `img_diff` must be subtracted
- 2) Copy the borders since the subimage is missing the borders to have a more readable comparison

See the difference?

→ Image subtraction (Part 2)

The proper shift

```
% Place the smaller image inside the larger image. Rotate the smaller image
% to comply with the convention that MATLAB® uses to display images.

img_diff(ij:-1:ij-size(img_subimage_gray,1)+1,ji:-1:ji-size(img_subimage_gray,2)+1) = ...
    rot90(img_subimage_gray,2) ;
```

```
% find the maximum
[ssr,snd] = max( crr(:) );
[ij,ji] = ind2sub(size(crr),snd);
```

rot90 Rotate array 90 degrees.

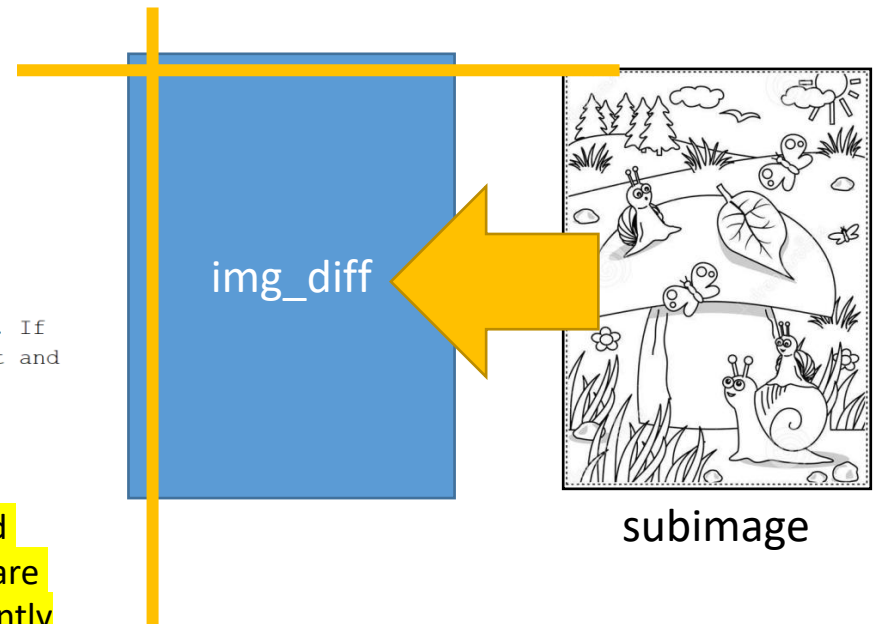
$B = \text{rot90}(A)$ is the 90 degree counterclockwise rotation of matrix A . If A is an N-D array, $\text{rot90}(A)$ rotates in the plane formed by the first and second dimensions.

$\text{rot90}(A,K)$ is the $K \times 90$ degree rotation of A , $K = +1, +2, \dots$

Example,

```
A = [1 2 3
      4 5 6]
B = rot90(A) = [ 3 6
                 2 5
                 1 4]
```

This notation is not in the exam!



Memory and
visualizations are
ordered differently

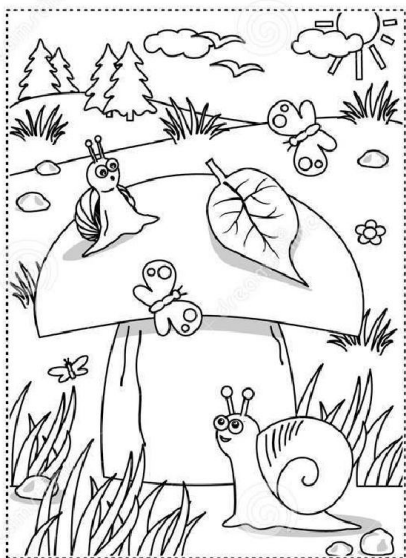
See the difference?

→ Image subtraction (check)

Open the previous figure by using the handle

```
figure(h1)
subplot(1,4,3)
imshow( img_diff )
title('just shift before subtraction')
```

template



subimage



just shift before subtraction



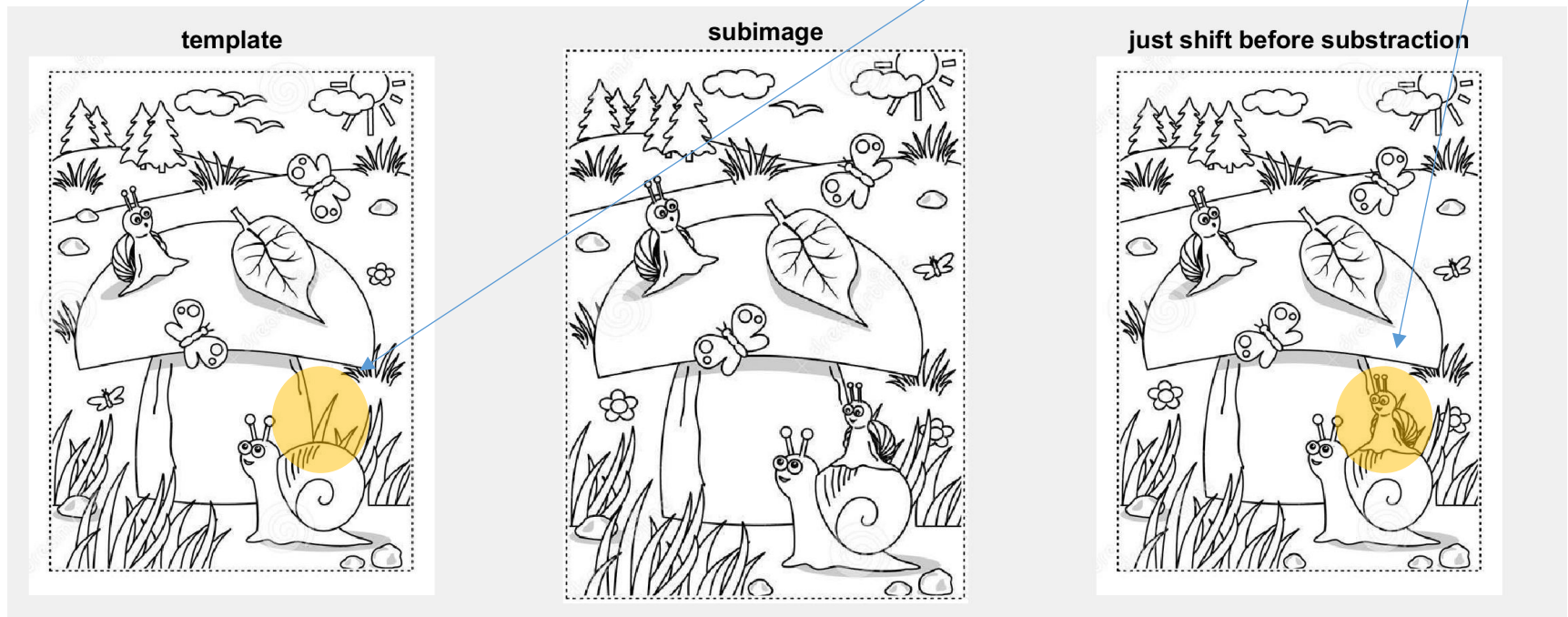
See the difference?

→ Image subtraction (check 1)

Open the previous figure by using the handle

```
figure(h1)  
subplot(1,4,3)  
imshow( img_diff )  
title('just shift before subtraction')
```

OK We have the subimage inside **img_diff**



See the difference?

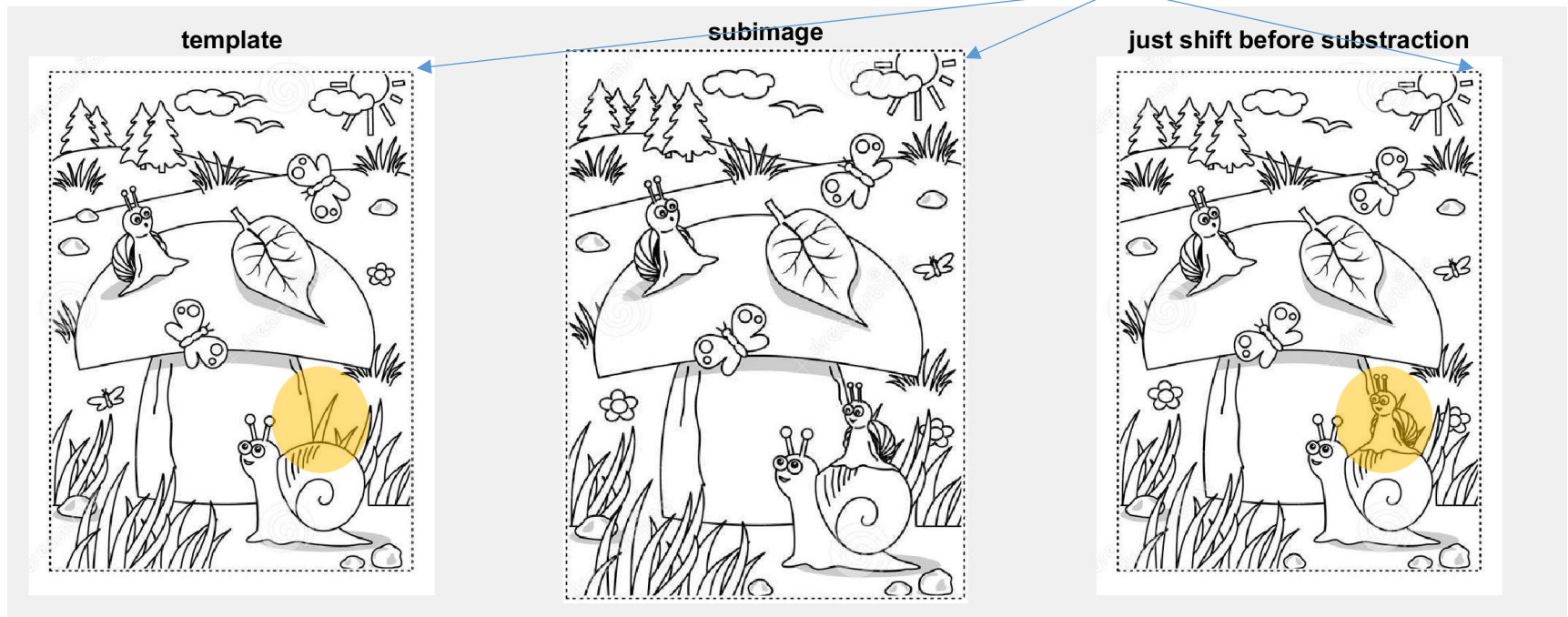
→ Image subtraction (check 2)

Open the previous figure by using the handle

```
figure(h1)
subplot(1,4,3)
imshow( img_diff )
title('just shift before subtraction')
```

OK We have the subimage inside

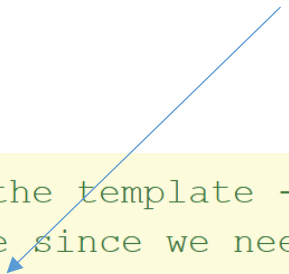
OK With the correct shifts



Creating the delta image

We need a double image not uint8!!

We need to be ready to store negative values in the delta!



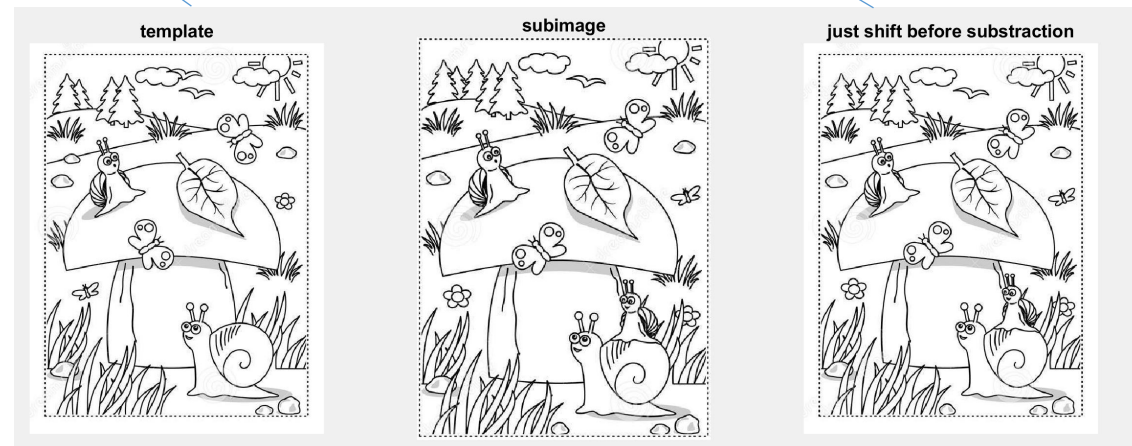
```
% img_diff2 = the template - the shifted subimage (with just the frame of template)
% we use double since we need negative values (not allowed in the uint8 format)
img_delta = double(zeros(size(img_template_gray)));
img_delta = double(img_template_gray) - double(img_diff);
```

Creating the delta image (2)

We need to be ready to store negative values in the delta!

```
% img_diff2 = the template - the shifted subimage (with just the frame of template)
% we use double since we need negative values (not allowed in the uint8 format)
img_delta = double(zeros(size(img_template_gray)));
img_delta = double(img_template_gray) - double(img_diff);
```

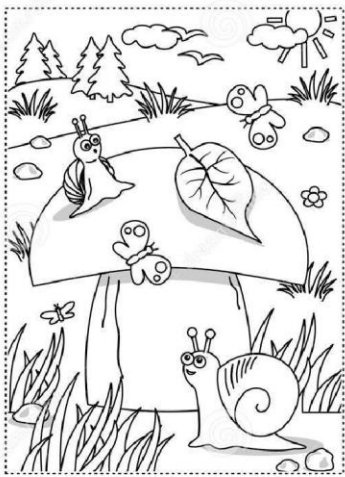
Make a «cast»!
To avoid any
conversion
problems!



Checking the delta image

```
% figure(h1)
subplot(1,4,4)
imshow( img_delta, [] )
title('shift and subtraction')
```

template



subimage



just shift before subtraction



shift and subtraction



Checking the delta image



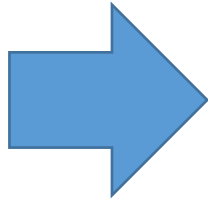
difference image



Improving the visualization

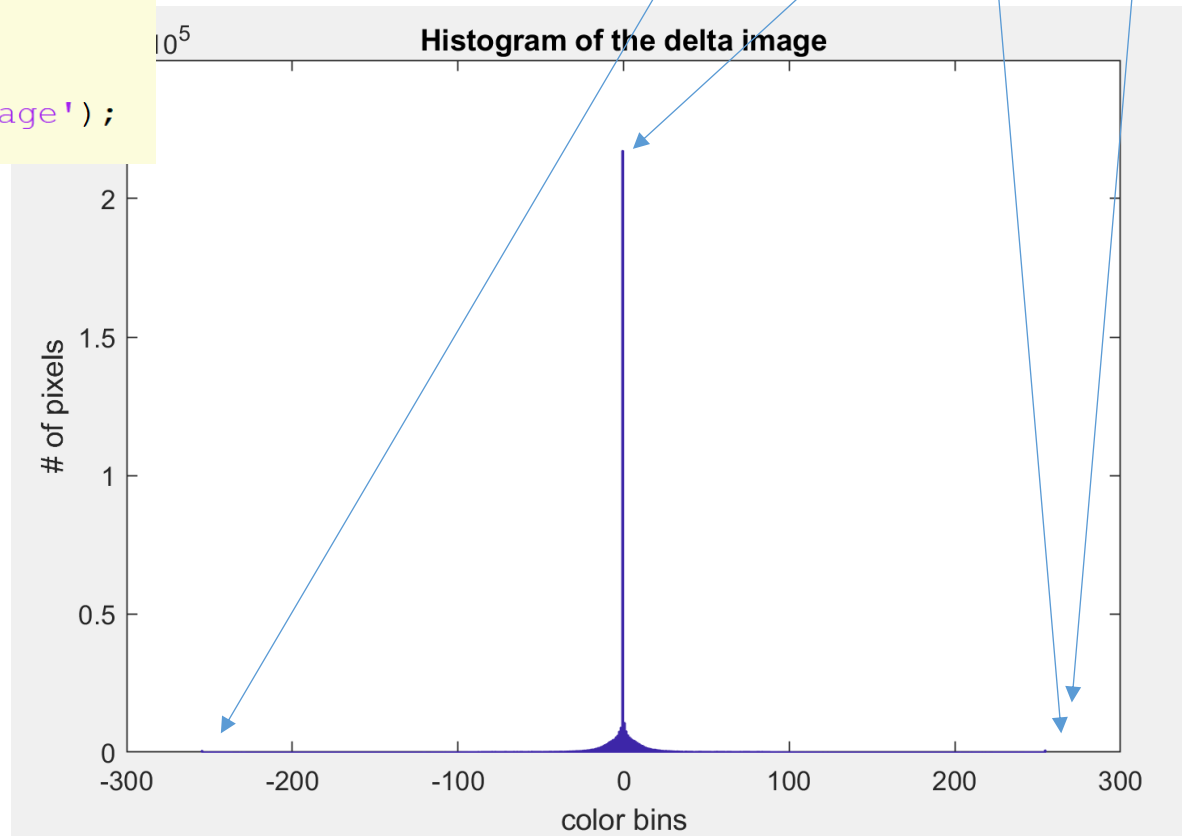
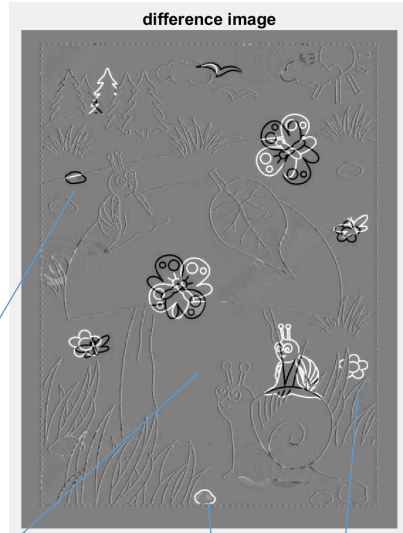
```
% improving the visualization
figure
subplot(1,2,1)
hist( img_delta(:), [2*255]);
xlabel('color bins')
ylabel('# of pixels')
title('Histogram of the delta image');
```

Understanding
how to use the
image histogram
in your work!



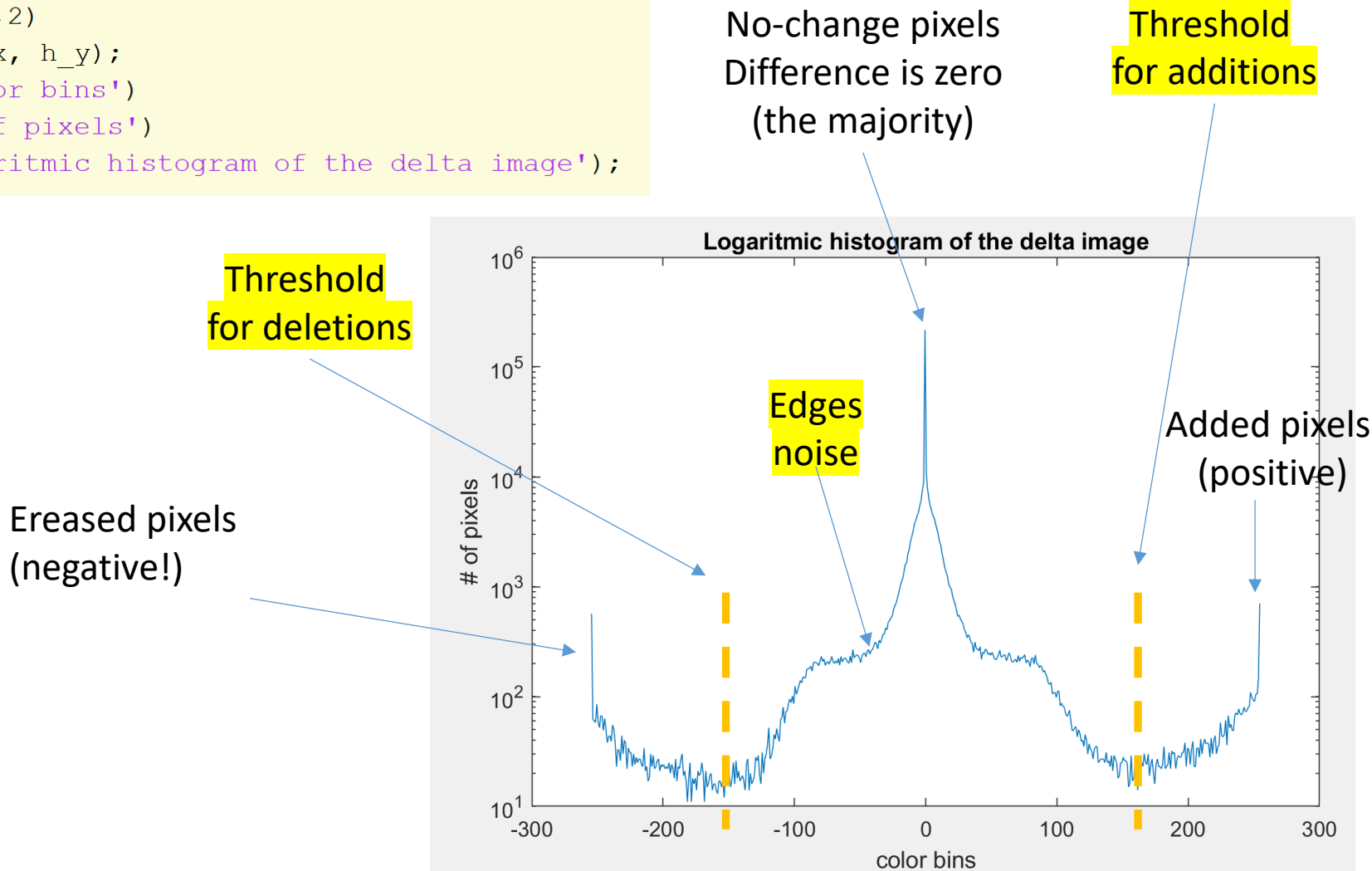
Sometimes histograms
are too “flat”
to really understand
what is happening

Zero values are grey
(histogram stretching)



Improving the visualization

```
subplot(1,2,2)
[h_y, h_x]= hist( img_delta(:), [2*255]);
subplot(1,2,2)
semilogy(h_x, h_y);
xlabel('color bins')
ylabel('# of pixels')
title('Logaritimic histogram of the delta image');
```



Separating the additions and deletions

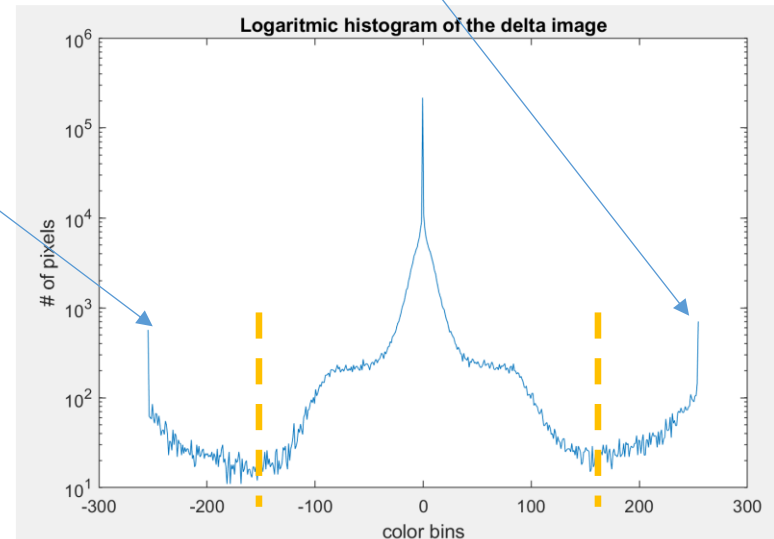
```
%% finding the good thresholds
```

```
max_value = max(img_delta(:));
```

```
min_value = min(img_delta(:));
```

```
additions = (img_delta > 0.5 * max_value) ;
```

```
deletions = (img_delta < 0.5 * min_value) ;
```



Separating the additions and deletions

```
%% finding the good thresholds
```

```
max_value = max(img_delta(:));
```

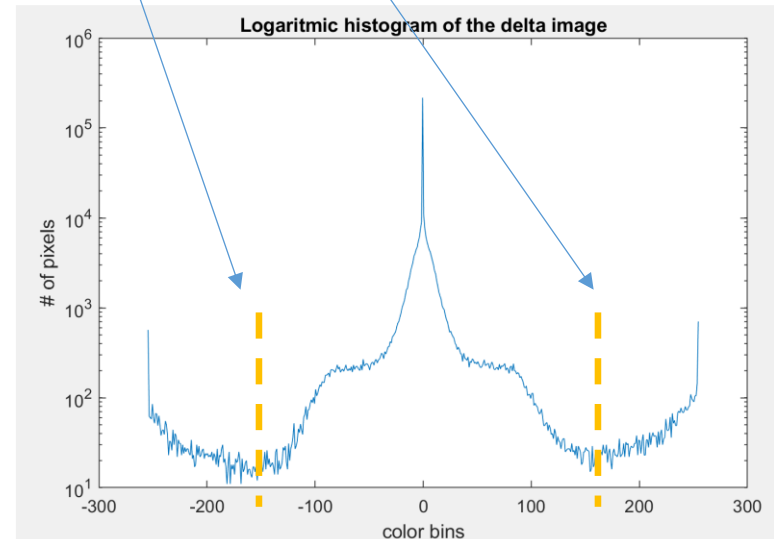
```
min_value = min(img_delta(:));
```

```
additions = (img_delta > 0.5 * max_value) ;
```

```
deletions = (img_delta < 0.5 * min_value) ;
```

```
>> whos additions
```

Name	Size	Bytes	Class
additions	736x555	408480	logical



Separating the additions and deletions

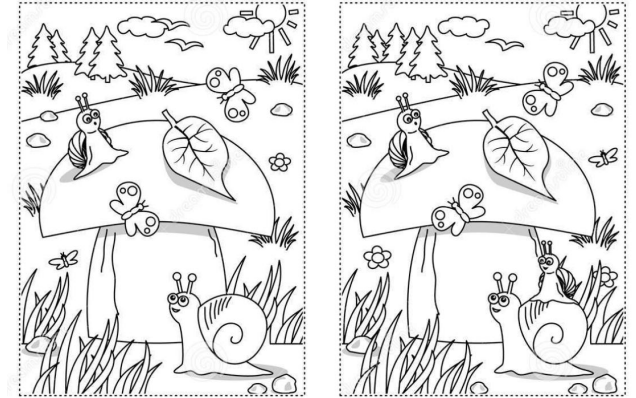
```
max_value = max(img_delta(:));
min_value = min(img_delta(:));

additions = (img_delta > 0.5 * max_value) ;
deletions = (img_delta < 0.5 * min_value) ;

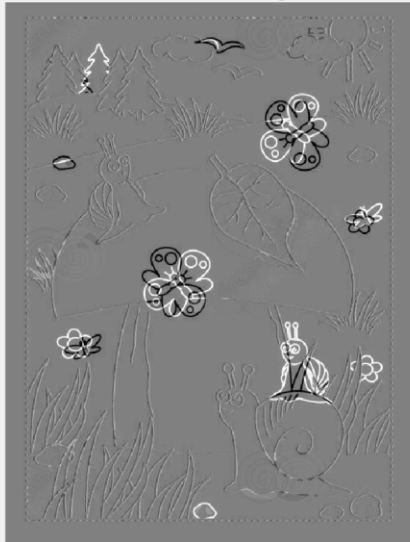
figure
subplot(1,3,1)
imshow( img_delta , [])
title('difference image')

subplot(1,3,2)
imshow( additions , [])
title('additions')

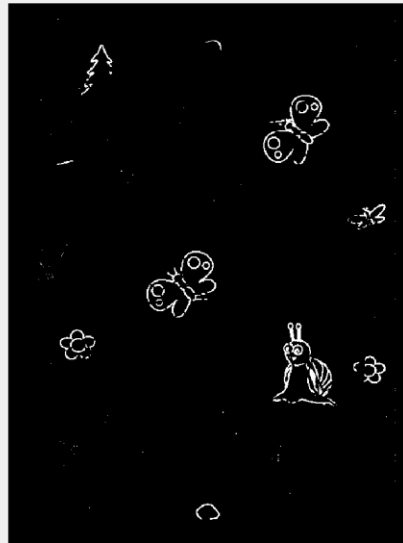
subplot(1,3,3)
imshow( deletions , [])
title('deletions')
```



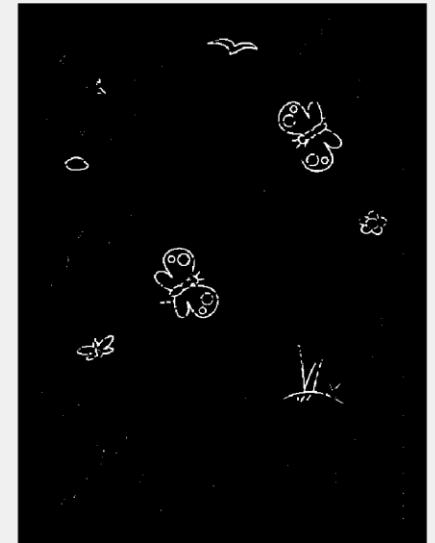
difference image



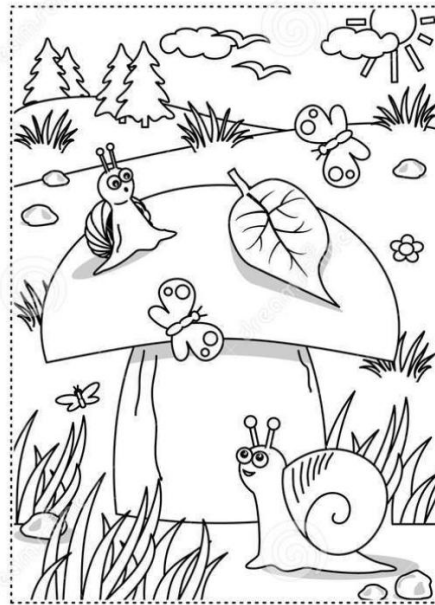
additions



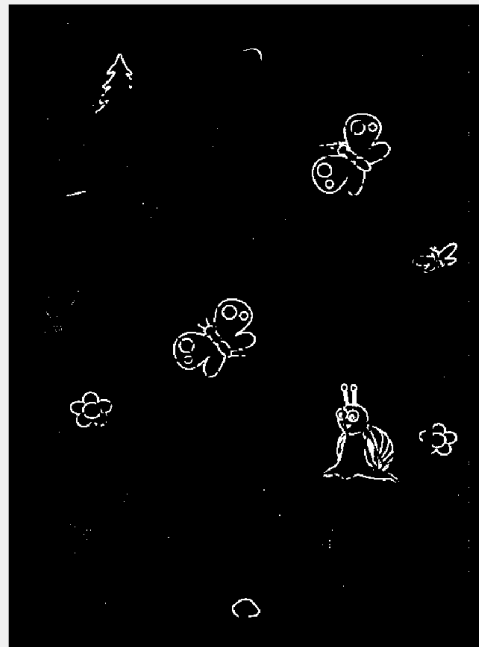
deletions



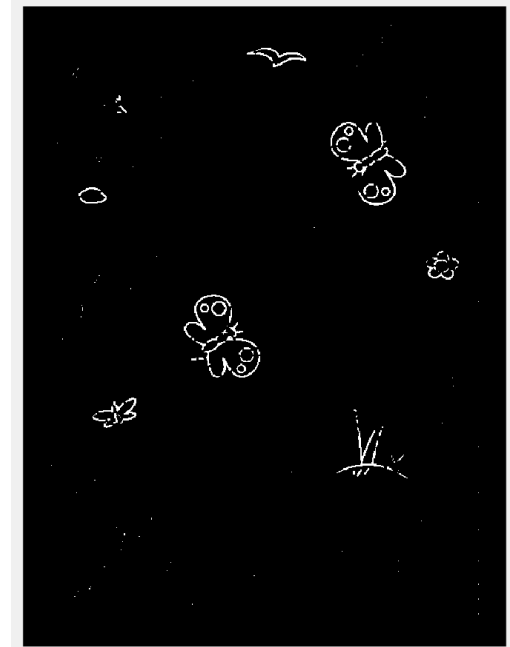
Overview



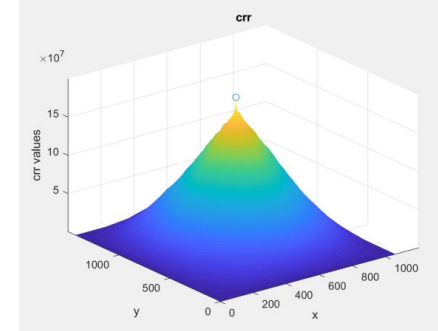
additions



deletions



Homeworks



- How is changing the CRR if you put in input the **same template image** as subimage
 - What about the maximum?
 - What about the shape of the CRR?
- How is changing the CRR if you put **two completely different images** in input?
 - What about the maximum?
 - What about the shape of the CRR?

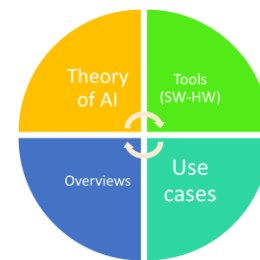


Toolboxes

Colab

Image retrieving

How to find images for your ML application



Creating an Image Retrieval System

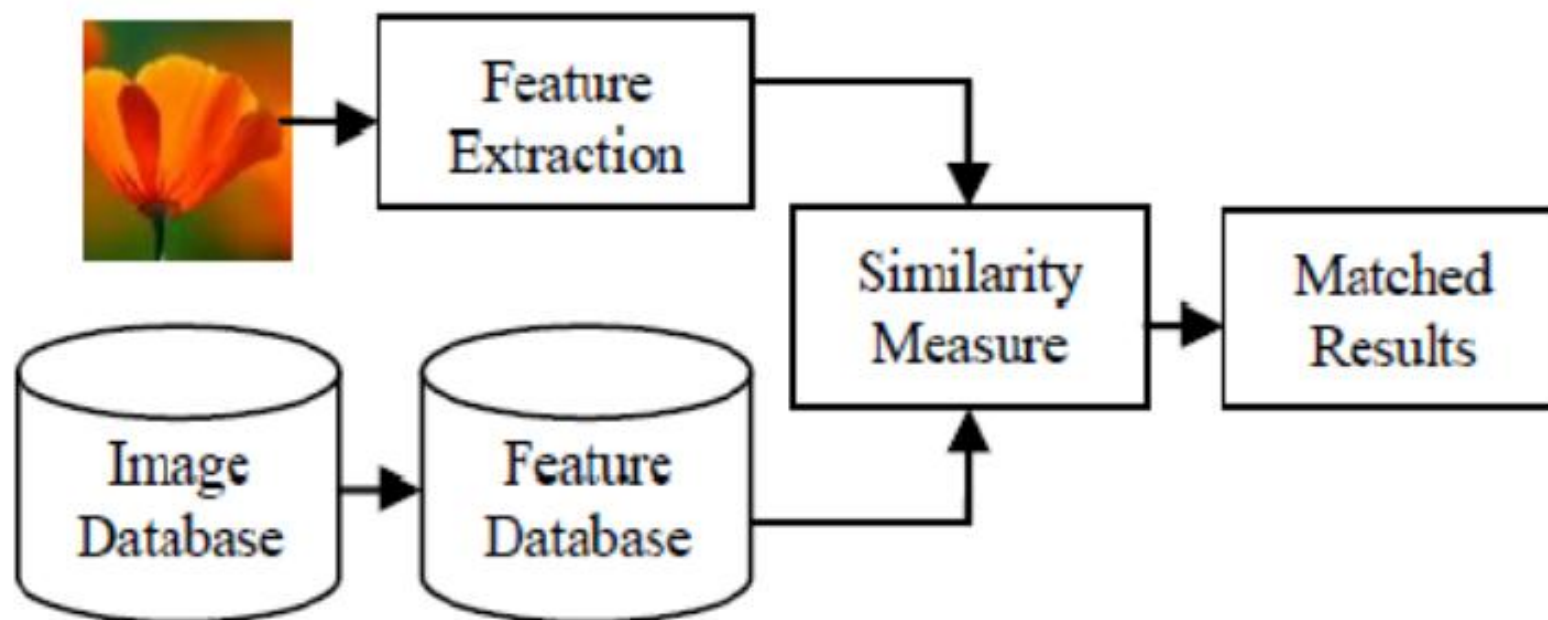
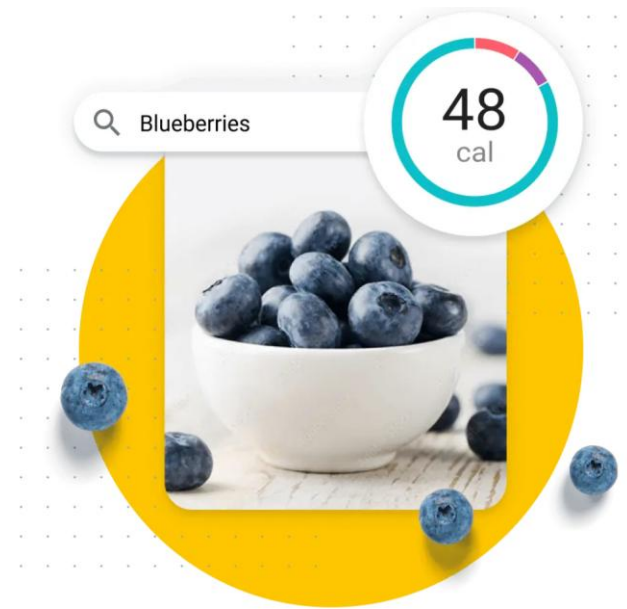


Image Retrieval System

Why?

- Think you want to create a new app counting the calories from the image of the dish
- You have to
 - Measure the quantities (gr)
 - Classify the foods or the recipes
- You can probably use NN/DeepLearning models to solve the task. You need to work on
 - Models
 - Topology
 - Datasets
- How to create a proper image dataset to train the model?

myfitnesspal™



Problem:

Find many pictures of a risotto

In a classical approach you can:

- download labelled image dataset
- take advantages from the competitors creating a new and complete image dataset by image retrieval and a proper labelling
- Use a basic repository link Google image search



Probably it's not enough → you have to

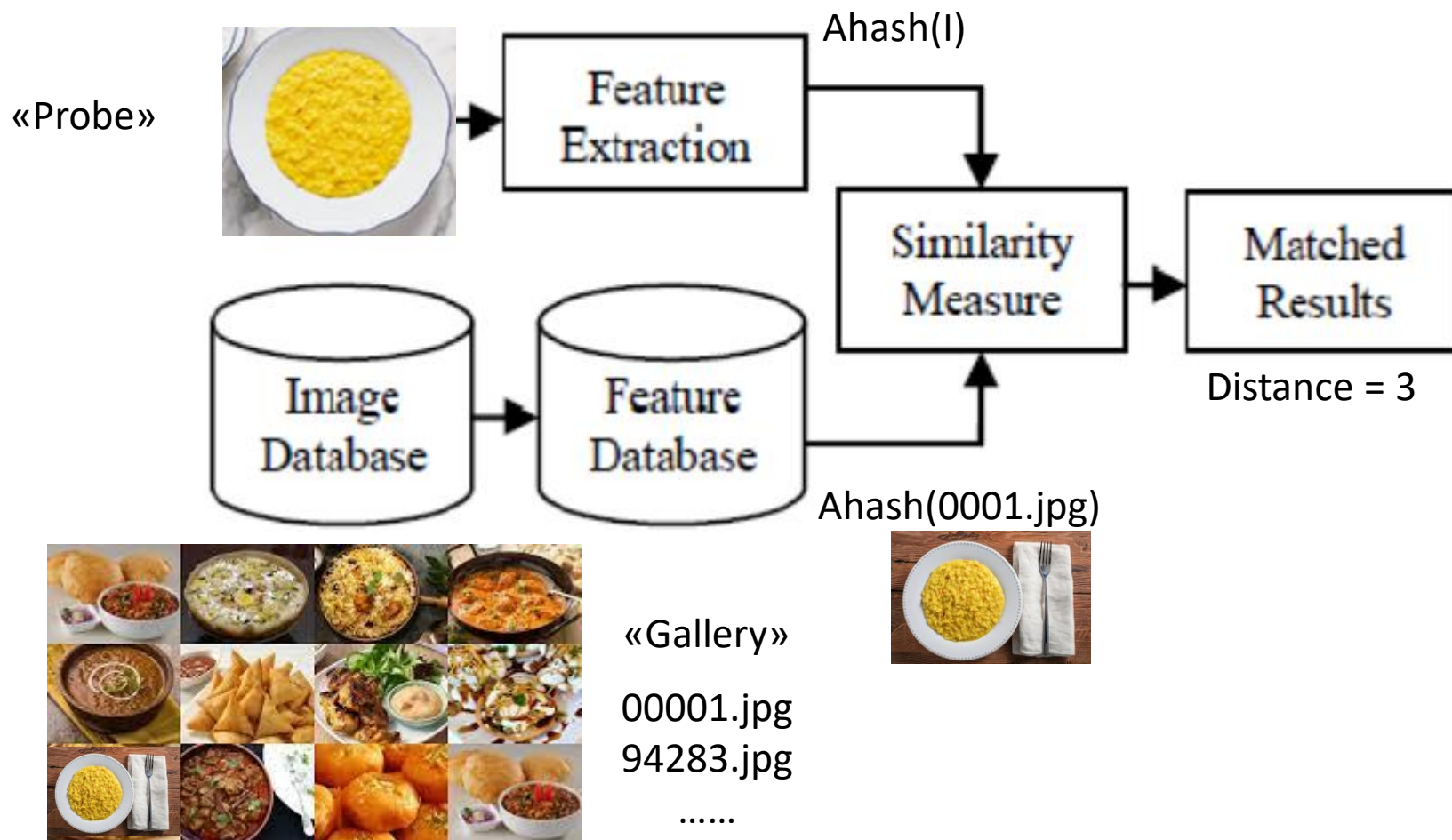
- access and very large dataset
- select the image you need
- make a careful labelling

You can do it automatically with an image retrieval system



Same thing for caws...
or in your next application!

Looking for the risotto images with image an image retrieval system



MEMO: Processing the image ahash

Fabio Scotti - Università degli Studi di Milano

Less.09

aHash, pHash, dHash

- 3 methods with similar initial steps
- Looking at average, perceptive and gradient difference and creating a binary feature
- aHash (a = Average)
 - 1) Reduce size (example: 8x8 R,G,B)
 - 2) Reduce to gray (8x8)
 - 3) Reduce to bits (0 if gray > below the mean gray)
 - 4) Merge the bits into a 64-bit integer. ← your hash

```
#----- AVERAGE HASH-----
hash1 = imagehash.average_hash(Image.open('dog1.jpg'))
hash2 = imagehash.average_hash(Image.open('dog2.jpg'))
hash3 = imagehash.average_hash(Image.open('dog3.jpg'))
hash4 = imagehash.average_hash(Image.open('dog4.jpg'))
hash5 = imagehash.average_hash(Image.open('panda.jpg'))
```

```
print(hash1)
print(hash2)
print(hash3)
print(hash4)
print(hash5)
```

```
0000d0e8ffffffff
00000b17ffffffff
0000f0ecffffffff
001040ecffdfdfdf
ffffe3e1c98181f3
```



Ranking features (a basic example)

- A ranking method (policy) on explainable features will permit to keep the image retrieval system usable and manageable
- Ex.: Policy: “pick the image file which is best ranking 1st, 2nd, or 3rd in at least 3 features”



Probe

ahash	dhash	colorhash	histogram distance	Features..
00001.Jpg dist = 3	23232.Jpg dist = 2	12122.Jpg dist = 3	00001.jpg dist = 2	23232.Jpg dist = 5
93894.jpg dist = 10	00001.jpg dist = 4	99094.jpg dist = 5	23423.Jpg dist = 3	232334.png dist = 7
34384.jpg dist = 13	34435.ipg dist = 8	34384.jpg dist = 7	9sd74.jpg dist = 6	93894.jpg dist = 9
...	...	...	...	...



00001.jpg

Winner of
the ranking!!!

Main points



- Cross-Correlation
- Matlab first steps with images
- Coding an example for similarity
 - Matlab
 - Python (image hash library)
- Image retrieval systems for specific image database creation